COURS & PROJET

MLOps : de l'expérimentation à la production

Concevoir, industrialiser et surveiller une plateforme de détection de fraude bancaire de bout en bout

Bassem Ben Hamed

Plateforme *Bank Fraud Detection MLOps*

Année universitaire 2026–2027

Plan du cours

1. Introduction au MLOps
2. Architecture cible de la plateforme
3. Cas d'usage : la fraude bancaire
4. Niveaux de maturité MLOps
5. Modélisation Machine Learning
6. Versionnement des données et du code
7. MLflow : tracking et registry
8. API et application Web
9. Testing : Pytest et Deepchecks
10. Conteneurisation avec Docker
11. CI/CD avec Jenkins
12. Orchestration avec Kubernetes
13. Monitoring : Prometheus, Grafana, Evidently
14. Synthèse et feuille de route

> Fil rouge du cours : **une seule application** suivie de l'EDA jusqu'au ré-entraînement automatique en production.

1

Introduction au MLOps

Le point de départ : un modèle n'est pas un produit

CE QUE FAIT UN DATA SCIENTIST

- Un notebook, un dataset figé, une métrique.
- Un `model.pkl` sur un portable.
- Un résultat : « *ROC-AUC = 0,999* ».

CE QU'ATTEND L'ENTREPRISE

- Une décision **en moins de 200 ms** sur chaque transaction.
- Une disponibilité **24/7**, tracée et auditable.
- Une performance qui **ne se dégrade pas** dans six mois.

Le fossé de la mise en production

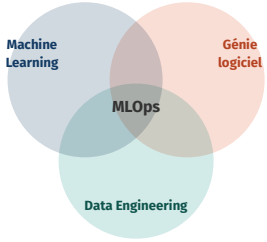
La majorité des modèles de Machine Learning développés en entreprise **n'atteignent jamais la production**. Quand ils y arrivent, le délai se compte en **mois**, et une part importante d'entre eux se dégrade silencieusement faute de surveillance.

Le MLOps est précisément la discipline qui **comble ce fossé**.

Qu'est-ce que le MLOps ?

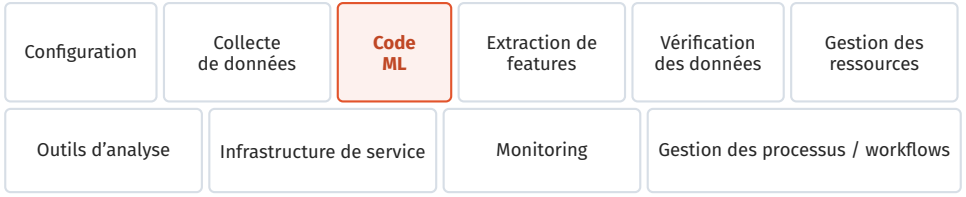
Définition

Le **MLOps** (*Machine Learning Operations*) est un ensemble de pratiques d'ingénierie qui vise à **déployer et maintenir de façon fiable et efficiente** des modèles de Machine Learning en production. Il se situe à l'intersection du **Machine Learning**, du **Génie logiciel** (DevOps) et de l'**Ingénierie des données**.



Le code ML n'est qu'une petite partie du système

D'après Sculley et al., *Hidden Technical Debt in Machine Learning Systems*, NeurIPS 2015



Dans un système ML réel, le code d'entraînement représente typiquement **5 à 10 %** du code total. Tout le reste est de l'**infrastructure**.

Ce cours porte sur les **90 % restants**.

La dette technique cachée des systèmes ML

DETTES DE DONNÉES

- **Dépendances de données instables** : une source amont change de format sans prévenir.
- **Dépendances sous-utilisées** : des features coûteuses qui n'apportent plus rien.
- **Boucles de rétroaction cachées** : le modèle influence les données qu'il recevra demain.

DETTES DE CODE ET DE SYSTÈME

- **Glue code** : 90 % de code de raccordement autour d'une librairie.
- **Pipeline jungles** : des scripts empilés que plus personne n'ose toucher.
- **Code expérimental mort** : des branches `if` laissées « au cas où ».
- **Anti-pattern CACE** : *Change Anything Change Everything*.

Conséquence pratique

La dette technique ML ne se voit pas dans les tests unitaires : elle se manifeste par une **érosion silencieuse de la performance métier**.

Logiciel classique et système ML : les différences

	Logiciel traditionnel	**Système de Machine Learning**
Comportement	Défini par des règles écrites	Induit à partir de données
Artefacts	Code source	Code + données + modèle
Tests	Déterministes (assert)	Statistiques (seuils, distributions)
Versionnement	Git suffit	Git + DVC + Model Registry
Cause de panne	Bug, exception	Dérive, données corrompues, biais
Dégradation	Le code ne « vieillit » pas seul	Le modèle **se périme**
Validation	Revue de code, CI	Revue + validation données + métriques
Rollback	Redéployer la version N−1	Redéployer **le bon couple** modèle/-preprocessing

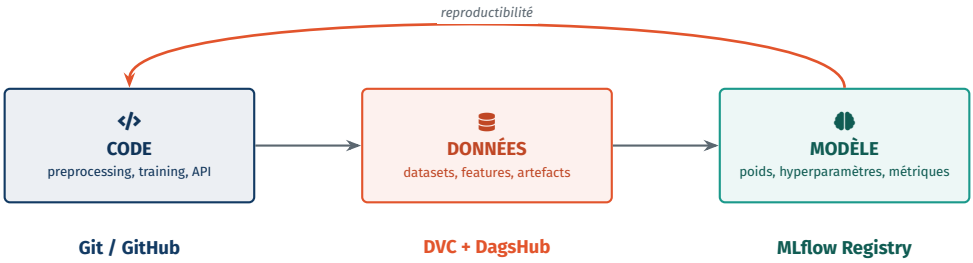
« Un modèle ML est le seul composant logiciel qui peut échouer sans qu'aucune ligne de code n'ait changé. »

DevOps et MLOps : ce qui change réellement

	DevOps	MLOps
Unité livrée	Une application	Un **pipeline** qui produit un modèle
Intégration (CI)	Compilation, tests unitaires	+ validation des **données** et du **modèle**
Livraison (CD)	Déploiement d'un binaire	Déploiement d'un **service d'inférence**
Nouveau pilier	—	**CT** : *Continuous Training*
Déclencheurs	Commit, tag	Commit, **nouvelles données**, **dérive**, calendrier
Surveillance	CPU, RAM, erreurs 5xx	+ qualité des prédictions, drift, biais
Équipe	Dev + Ops	Dev + Ops + Data Scientist + Data Engineer

Le MLOps **n'est pas un remplacement** du DevOps : c'est le DevOps augmenté d'une dimension **données** et d'une dimension **modèle**.

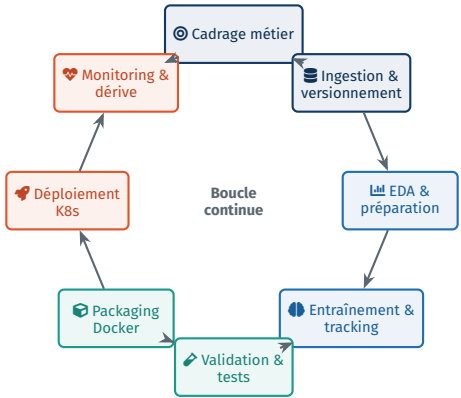
Les trois artefacts à versionner



Règle d'or de la reproductibilité

Une **expérience reproductible** = un commit Git **+** une version de données **+** un environnement figé. Si l'un des trois manque, le résultat n'est pas reproductible, il est *anecdotique*.

Le cycle de vie MLOps



La sortie du monitoring **re-alimente** l'ingestion : c'est ce bouclage qui distingue le MLOps d'un simple projet de data science.

Qui fait quoi ? Les rôles autour du pipeline

Rôle	Responsabilité principale	Outils du cours
Data Engineer	Collecte, qualité et disponibilité des données	DVC, DagsHub, Deepchecks
Data Scientist	Feature engineering, modélisation, métriques	Pandas, LightGBM, MLflow
ML Engineer	Industrialisation du pipeline, packaging	Docker, MLflow, FastAPI
DevOps / SRE	CI/CD, infrastructure, disponibilité	Jenkins, Kubernetes
Ingénieur monitoring	Observabilité, alerting, dérive	Prometheus, Grafana, Evidently
Métier / Risque	Définition du coût d'erreur, seuil de décision	KPIs métier

Dans un projet de taille réduite (comme le nôtre), **une même personne porte plusieurs rôles** — raison de plus pour que le pipeline soit automatisé et documenté.

CI, CD et CT : les trois boucles d'automatisation

CI — Intégration continue

Chaque modification est **validée automatiquement** :

- lint et tests unitaires ;
- validation des données ;
- test du pipeline complet ;
- construction des images.

CD — Livraison continue

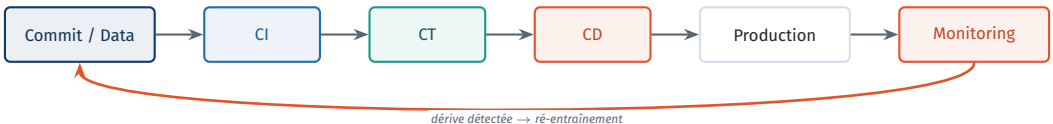
Le livrable validé est **déployé sans intervention manuelle** :

- push des images Docker ;
- mise à jour des *Deployments* ;
- rolling update sans coupure ;
- vérification du *rollout*.

CT — Entraînement continu

Spécifique au ML : le modèle est **ré-entraîné automatiquement** quand

- de nouvelles données arrivent ;
- une **dérive** est détectée ;
- la performance chute ;
- une échéance est atteinte.



Les principes fondamentaux du MLOps

1. **Reproductibilité** — toute exécution doit pouvoir être rejouée à l'identique.
2. **Automatisation** — tout ce qui est fait deux fois à la main doit être scripté.
3. **Versionnement intégral** — code, données, modèles, environnements, configurations.
4. **Tests à tous les étages** — données, features, modèle, API, infrastructure.
5. **Traçabilité** — pour toute prédiction, on doit pouvoir remonter au modèle, au run et au dataset d'origine.
6. **Monitoring continu** — technique *et* métier *et* statistique.
7. **Déploiement progressif et réversible** — rolling update, rollback en un instant.
8. **Collaboration** — un langage et des artefacts communs entre data et ops.

Dans notre projet

Ces huit principes sont notre grille de lecture : chaque section du cours ajoute une brique qui en implémente concrètement un ou plusieurs.

Les défis propres à un système ML en production

DÉFIS LIÉS AUX DONNÉES

- **Data drift** : la distribution des entrées change.
- **Concept drift** : la relation entrée–sortie change (les fraudeurs s'adaptent).
- **Training–serving skew** : le preprocessing d'entraînement diffère de celui d'inférence.
- **Déséquilibre extrême** des classes.

DÉFIS LIÉS AU SYSTÈME

- **Latence** : décider en quelques dizaines de millisecondes.
- **Scalabilité** : absorber les pics de trafic.
- **Reproductibilité** des environnements.
- **Gouvernance** : auditabilité, explicabilité, conformité.

Le piège du training–serving skew

Le modèle est excellent en test et médiocre en production, simplement parce que l'encodage ou la normalisation diffère. **Parade** : sérialiser les *processeurs* et les réutiliser tels quels à l'inférence.

Objectifs pédagogiques du cours

À l'issue de ce cours, vous saurez

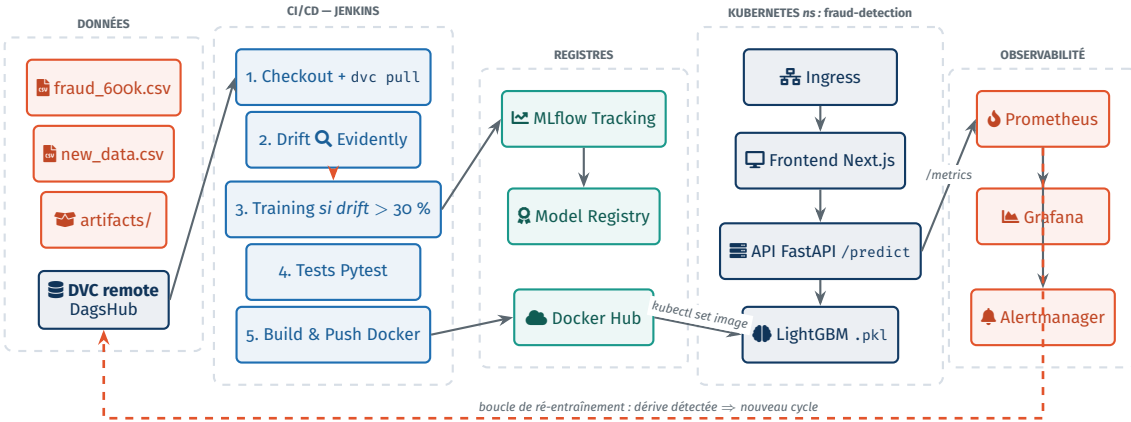
1. Positionner une organisation sur une **échelle de maturité MLOps** et construire son plan de progression.
2. Versionner conjointement **code et données** avec Git et DVC.
3. Tracer des expériences et gouverner des modèles avec **MLflow**.
4. Exposer un modèle via une **API FastAPI** et une application Web Next.js.
5. Écrire des tests **Pytest** et des suites de qualité de données **Deepchecks**.
6. Conteneuriser proprement avec **Docker** multi-stage.
7. Automatiser CI/CD/CT avec **Jenkins**.
8. Déployer et exploiter sur **Kubernetes**.
9. Surveiller avec **Prometheus, Grafana et Evidently AI**, et déclencher un ré-entraînement automatique.

Le tout illustré sur un **cas d'usage unique et complet** : la détection de fraude bancaire.

2

Architecture cible de la plateforme

L'architecture que nous allons construire



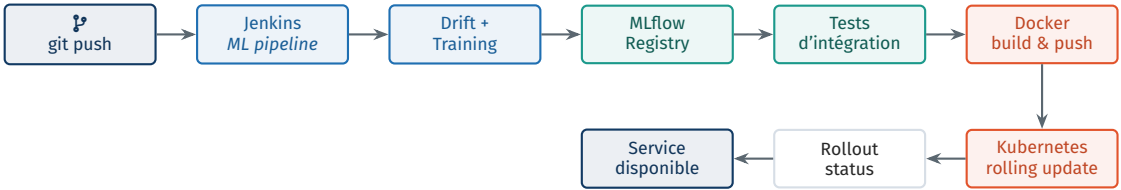
Les cinq couches de la plateforme

	Couche	Rôle	Technologies
1	Données	Stocker et versionner datasets et artefacts	DVC, DagsHub
2	CI/CD	Orchestrer drift, entraînement, tests, build	Jenkins
3	Registres	Gouverner modèles et images	MLflow, Docker Hub
4	Service	Servir les prédictions et l'interface	FastAPI, Next.js, Kubernetes
5	Observabilité	Mesurer, alerter, détecter la dérive	Prometheus, Grafana, Evidently

Principe de conception

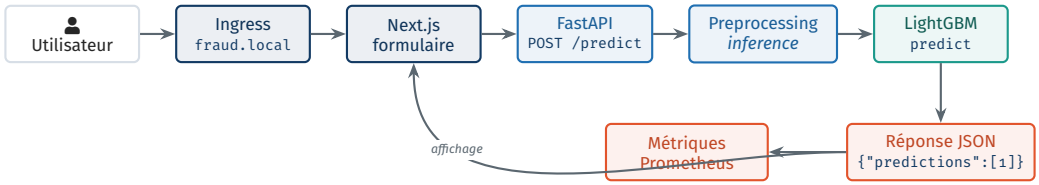
Chaque couche est **indépendamment remplaçable** : on peut échanger Jenkins contre GitLab CI, DagsHub contre S3, ou Kubernetes contre un service managé, **sans réécrire les autres couches**. C'est le bénéfice d'interfaces explicites (fichiers versionnés, images OCI, endpoints HTTP).

Flux nº 1 — Du commit à la production



- Aucune étape manuelle entre le `git push` et le pod qui répond.
- Chaque étape peut **échouer et arrêter la chaîne** : c'est le rôle des *quality gates*.
- Le déploiement est **progressif** : l'ancien pod ne meurt qu'une fois le nouveau prêt.

Flux nº 2 — De la transaction à la décision

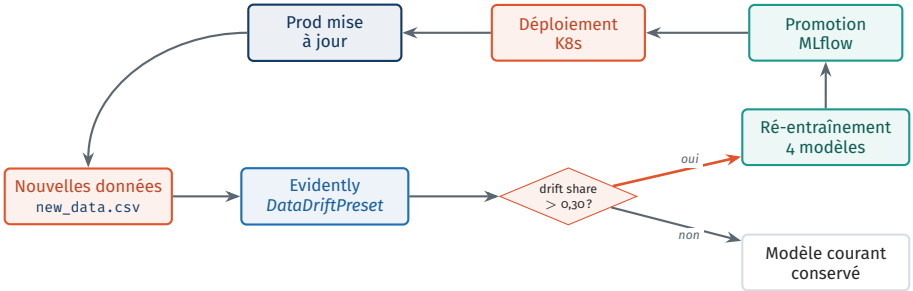


CE QUE LE CHEMIN DOIT GARANTIR

- Le **même preprocessing** qu'à l'entraînement.
- Une latence mesurée à chaque appel (histogramme).
- Un compteur de prédictions par classe.

Chaque prédiction produit **deux** sorties : la réponse métier *et* un signal d'observabilité.

Flux nº 3 — La boucle de ré-entraînement automatique



C'est le cœur du MLOps de niveau 2

Aucun humain n'intervient dans cette boucle. Le déclencheur n'est pas un développeur, c'est **un signal statistique mesuré sur les données de production**.

La pile technologique retenue

Besoin	Outil retenu	Pourquoi	Alternatives
Version. code	Git / GitHub	Standard universel	GitLab, Bitbucket
Version. données	DVC + DagsHub	Syntaxe Git-like, remote gratuit, UI intégrée	LakeFS, Delta Lake, Git LFS
Tracking XP	MLflow	Open source, agnostique, registry inclus	W&B, Neptune, Comet
Modélisation	LightGBM	Rapide, robuste au déséquilibre	XGBoost, CatBoost
API	FastAPI	Async, typage Pydantic, OpenAPI auto	Flask, BentoML, KServe
Interface	Next.js (React)	SSR, App Router, écosystème riche	Streamlit, Gradio
Tests	Pytest + Deepchecks	Unitaires *et* qualité des données	unittest, Great Expectations
Packaging	Docker multi-stage	Images légères et reproductibles	Podman, Buildpacks
CI/CD	Jenkins	Auto-hébergé, plugins, contrôle total	GitHub Actions, GitLab CI
Orchestration	Kubernetes	Standard, auto-guérison, rolling update	Docker Swarm, Nomad
Métriques	Prometheus + Grafana	Modèle pull, PromQL, dashboards	Datadog, InfluxDB
Dérive	Evidently AI	Rapports HTML, presets prêts à l'emploi	NannyML, Alibi Detect

Trois environnements, un seul code

	Local (dév.)	CI (Jenkins)	Cluster (prod)
Exécution	`docker-compose up`	Agent Jenkins + venv	Pods Kubernetes
Données	`dvc pull` partiel	`dvc pull` ciblé	Artefacts dans l'image
Modèle	`.pkl` local	Téléchargé de MLflow	Copié dans l'image
URL API	`localhost:8000`	`192.168.49.2:30080`	`api.fraud.local`
Objectif	Itérer vite	Valider et construire	Servir de façon fiable

Le piège classique

Si la configuration diffère entre environnements autrement que par des **variables d'environnement**, on obtient le fameux « *ça marche sur ma machine* ». D'où l'usage systématique de `ConfigMap`, de `ARG`/`ENV` Docker et de `.env`.

Organisation du dépôt

```
.
|-- .dvc/                       # configuration DVC (remote DagsHub)
|-- data.dvc  artifacts.dvc     # pointeurs versionnes (hash MD5) -> donnees & artefacts
|-- notebooks/                  # 01_eda  02_preprocessing  03_modeling  04_mlflow_tracking
|-- backend/
|   |-- src/api.py              # API FastAPI : /predict /predictCSV /metrics /drift-status
|   |-- src/preprocessing.py    # classe PreprocessingFraud (train + inference)
|   `-- Dockerfile              # image multi-stage python:3.12-slim
|-- frontend/                   # application Next.js (App Router) + Dockerfile 3 etages
|-- jenkins/
|   |-- Jenkinsfile.ml.txt        # pipeline drift -> train -> register -> test
|   |-- Jenkinsfile.deploy.txt    # pipeline build -> push -> deploy K8s
|   |-- drift_detection.py        # Evidently AI
|   |-- train_model.py            # entrainement continu + MLflow
|   `-- register_best_model.py    # promotion du meilleur modele
|-- k8s/                        # namespace, deployments, services, ingress, monitoring
|-- test/                       # test_api_mock.py (Pytest) + test_fraud_scenario.py
|-- testing/deepchecks.ipynb    # suites de qualite des donnees
`-- docker-compose.yml          # execution locale multi-services
```

Ce que cette architecture apporte concrètement

Bénéfices

- Un modèle mis à jour **sans intervention humaine**.
- Une traçabilité complète : run MLflow $\rightarrow$ commit $\rightarrow$ image $\rightarrow$ pod.
- Un retour arrière en une commande.
- Une détection de dégradation **avant** la plainte client.
- Des environnements identiques du portable au cluster.

Coûts à assumer

- Complexité opérationnelle réelle (Kubernetes, Jenkins).
- Coût d'infrastructure permanent.
- Compétences pluridisciplinaires nécessaires.
- Temps de mise en place initial important.

Ce coût ne se justifie que si le modèle est **critique et amené à évoluer**. La section suivante donne la grille pour décider.

3

Cas d’usage : la détection de fraude bancaire

Le problème métier

LE CONTEXTE

Une banque traite des **millions de transactions** par carte chaque jour. Une fraction infime est frauduleuse, mais chaque fraude non détectée représente une perte sèche, un litige et une atteinte à la confiance.

LA MISSION

- Décider, **pour chaque transaction**, si elle est légitime ou frauduleuse.
- En **temps réel**, avant l'autorisation du paiement.
- Avec un **taux de fausses alertes maîtrisé** : bloquer un client honnête coûte cher aussi.

Trois difficultés structurelles

1. **Déséquilibre extrême** : moins de 1 % de fraudes.
2. **Adversarialité** : les fraudeurs changent de mode opératoire.
3. **Contrainte temps réel** : quelques dizaines de millisecondes par décision.

La difficulté nº 2 est exactement ce qu'on appelle du **concept drift** : elle justifie à elle seule tout le pipeline MLOps.

Le coût des erreurs : une asymétrie fondamentale

		Prédiction du modèle : Légitime	Prédiction du modèle : Fraude
Réalité	Légit.	**VN** la transaction passe	**FP** client bloqué à tort
Réalité	Fraude	**FN** **fraude non détectée**	**VP** fraude bloquée

COÛT D'UN FAUX NÉGATIF (FN)

Montant de la transaction, frais de litige, remboursement, réputation. **Élevé et direct.**

COÛT D'UN FAUX POSITIF (FP)

Client mécontent, appel au service client, risque d'attrition. **Modéré mais massif** si le taux est élevé.

Le **seuil de décision** n'est pas un choix technique : c'est un **arbitrage métier**. Baisser le seuil à 0,3 augmente le rappel (moins de fraudes ratées) au prix de plus de faux positifs. Ce paramètre doit être exposé, versionné et discuté avec le métier.

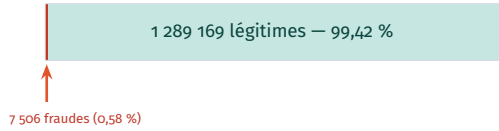
Le jeu de données

VOLUMÉTRIE

- **1 296 675** transactions (année 2019–2020).
- **23** colonnes brutes, $\approx$ 1 Go en mémoire.
- **Aucune** valeur manquante.
- Un sous-ensemble `fraud_600k.csv` sert de **référence** pour la détection de dérive.

VARIABLE CIBLE

`is_fraud` $\in \{0, 1\}$



Ratio 1 : 171

Un modèle qui prédit *« jamais de fraude »* obtient **99,42 % d'accuracy**. C'est le piège classique : **l'accuracy est inutilisable ici**.

Dictionnaire des variables

Colonne	Type	Description	Intérêt pour la fraude
`trans_date_trans_time`	datetime	Horodatage de la transaction	Heure, jour, week-end
`cc_num`	int	Numéro de carte (pseudonymisé)	Historique par carte
`merchant`	string	Commerçant	Commerçants à risque
`category`	catégorie	14 catégories d'achat	**Discriminant fort** (`misc_net`, `shopping_net`)
`amt`	float	Montant en dollars	**Très discriminant**
`lat`, `long`	float	Position du domicile	Base du calcul de distance
`merch_lat`, `merch_long`	float	Position du commerçant	Base du calcul de distance
`city_pop`	int	Population de la ville	Contexte géographique
`dob`	date	Date de naissance	Permet de calculer l'âge
`is_fraud`	binaire	**Cible**	—

Colonnes écartées à la modélisation : `first`, `last`, `street`, `trans_num`, `unix_time` (identifiants sans pouvoir prédictif, et porteuses de données personnelles).

Ce que révèle l'analyse exploratoire

SIGNAUX EXPLOITABLES

- **Montant** : les fraudes se concentrent sur des montants atypiques ; le 99e centile est à \$546, le maximum observé à \$28 949.
- **Heure** : forte surreprésentation des transactions **nocturnes (0h–3h)**.
- **Catégorie** : `misc_net`, `shopping_net` et `grocery_pos` sur-représentées.
- **Distance domicile–commerçant** : les écarts géographiques anormaux sont un signal fort.
- **Âge** du porteur au moment de la transaction.

CONSÉQUENCES POUR LE PIPELINE

- Ces signaux ne sont **pas dans les données brutes** : il faut les **construire** (feature engineering).
- Les mêmes transformations devront être rejouées **à l'identique** en production → classe partagée `PreprocessingFraud`.
- Les colonnes surveillées pour la dérive seront `amt`, `category`, `gender`, `city_pop`, `merch_lat`, `merch_long`.

Dans notre projet

L'EDA n'est pas une étape décorative : elle **définit** les features à créer et les colonnes à monitorer en production.

Quelles métriques pour un problème déséquilibré ?

Métrique	Formule	Lecture métier
Accuracy	$\frac{VP+VN}{\text{total}}$	✗ Trompeuse : 99,4 % sans rien apprendre
Précision	$\frac{VP}{VP+FP}$	Parmi les alertes, combien sont de vraies fraudes
Rappel	$\frac{VP}{VP+FN}$	✓ Parmi les fraudes, combien sont détectées
F1-score	$2 \cdot \frac{P \cdot R}{P+R}$	Compromis précision/rappel
ROC-AUC	Aire sous la courbe TPR/FPR	Capacité de séparation, tous seuils confondus
PR-AUC	Aire sous précision/rappel	✓ Plus fiable que ROC-AUC en cas de fort déséquilibre

Notre choix

Sélection du modèle sur la **PR-AUC de validation** (fiable en cas de fort déséquilibre, jamais calculée sur le test). Le seuil de décision est celui qui **maximise le F1 sur la validation** ; il est écrit dans `metadata.json` et appliqué par l'API. La variable `FRAUD_THRESHOLD` permet de le surcharger sans reconstruire l'image.

Contraintes non fonctionnelles

Contrainte	Cible	Traduction technique
Latence d'inférence	P99 $<$ 2 s (alerte)	Histogramme Prometheus + *PrometheusRule*
Disponibilité	Pas de coupure au déploiement	Rolling update K8s + *readinessProbe*
Auto-guérison	Redémarrage automatique	*livenessProbe* sur `/health`
Scalabilité	Absorber les pics	`replicas`, `requests`/`limits`
Traçabilité	Retrouver le modèle d'une prédiction	MLflow run_id + `metadata.json`
Confidentialité	Pas de PII en clair	Colonnes nominatives écartées
Fraîcheur du modèle	Ré-entraînement si dérive	Pipeline CT déclenché par Evidently

Chacune de ces lignes se traduit par un **objet concret** dans le dépôt : une sonde, une règle d'alerte, un fichier de métadonnées. C'est cela, « industrialiser ».

Les indicateurs suivis en production

Métrique Prometheus	Type	Ce qu'elle raconte
`fraud_predictions_total{result}`	Counter	Volume de prédictions par classe
`fraud_rate_current`	Gauge	Taux de fraude instantané — alerte si > 30 %
`fraud_prediction_latency_seconds`	Histogram	Latence du modèle (quantiles P50/P95/P99)
`fraud_prediction_errors_total`	Counter	Erreurs d'inférence
`fraud_drift_score`	Gauge	Part de features en dérive (Evidently)
`fraud_drift_detected`	Gauge	1 si dérive au-delà du seuil
`fraud_model_loaded`	Gauge	Le modèle est-il bien chargé dans le pod
`fraud_model_version_info`	Gauge	Version et nom du modèle servi
`fraud_jenkins_build_status`	Gauge	Résultat du dernier pipeline ML

On distingue trois familles : **métriques techniques** (latence, erreurs), **métriques métier** (taux de fraude) et **métriques ML** (dérive, version). Un tableau de bord complet les affiche côte à côte.

Périmètre, hypothèses et limites

Dans le périmètre

- Classification binaire par transaction.
- Inférence unitaire et batch (CSV).
- Ré-entraînement déclenché par la dérive.
- Déploiement sur un cluster Kubernetes local (Minikube).
- Monitoring technique, métier et statistique.

Hors périmètre / limites

- Pas de *feature store* : pas d'agrégats temporels par carte (nombre de transactions dans la dernière heure, etc.).
- Pas de *ground truth* temps réel : les vraies étiquettes de fraude arrivent avec des semaines de retard.
- Pas d'explicabilité individuelle (SHAP) exposée à l'utilisateur.
- Données synthétiques : les conclusions métier ne sont pas transposables telles quelles.

Point important : sans étiquettes en temps réel, on ne peut pas mesurer directement la performance en production. C'est pourquoi on surveille des **proxys** : dérive des entrées et taux de fraude prédit.

4

Les niveaux de maturité MLOps

Pourquoi raisonner en niveaux de maturité ?

LE BESOIN

« Faire du MLOps » n'est pas binaire. Une équipe peut très bien versionner ses données sans savoir déployer, ou déployer sans rien surveiller. Un **modèle de maturité** sert à :

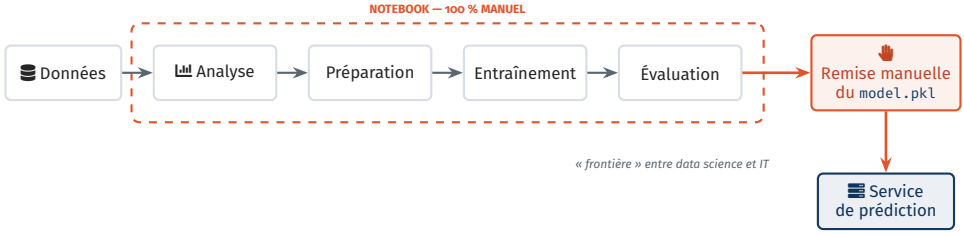
- établir un **diagnostic** objectif de l'existant ;
- identifier le **prochain goulot d'étranglement** ;
- justifier un investissement auprès de la direction ;
- éviter de sur-outiller un besoin simple.

Les deux référentiels de référence

- **Google Cloud** — 3 niveaux (0, 1, 2), centrés sur l'**automatisation du pipeline**.
- **Microsoft Azure** — 5 niveaux (0 à 4), centrés sur la **maturité organisationnelle**.

Monter d'un niveau a un **coût réel**. L'objectif n'est pas d'atteindre le niveau maximal, mais le **niveau adapté à l'enjeu**.

Google — Niveau 0 : processus entièrement manuel



CARACTÉRISTIQUES

- Tout se passe dans un notebook, exécuté à la main.
- Séparation étanche entre l'équipe ML et l'équipe IT.
- Livraison du modèle : un fichier, un e-mail.
- Aucune CI, aucune CD, aucun monitoring ML.

SYMPTÔMES À RECONNAÎTRE

- « Quel notebook a produit ce modèle ? »
- Une mise en production tous les six mois.
- Le modèle en prod n'est jamais ré-entraîné.
- Personne ne sait si sa performance a chuté.

Niveau 0 : pourquoi c'est intenable en production

Les cinq faiblesses

1. **Non reproductible** : l'ordre d'exécution des cellules influence le résultat, l'environnement n'est pas figé.
2. **Non testé** : aucun garde-fou avant la production.
3. **Training–serving skew** garanti : le preprocessing est réécrit à la main côté service.
4. **Aucune fraîcheur** : le modèle se périme sans alerte.
5. **Facteur bus = 1** : une seule personne sait faire tourner la chaîne.

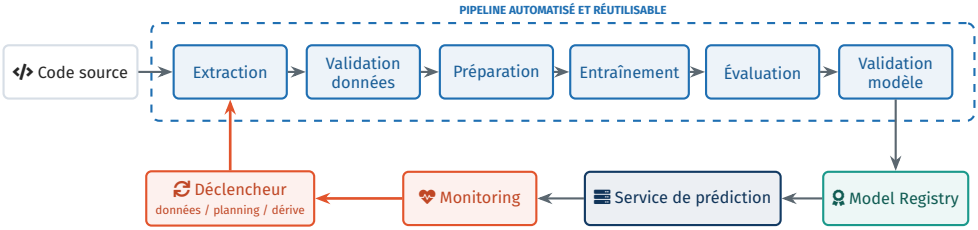
QUAND LE NIVEAU 0 EST ACCEPTABLE

- Preuve de concept, étude ponctuelle.
- Modèle utilisé une fois puis jeté.
- Aucun enjeu de disponibilité ni de fraîcheur.

LE PREMIER PAS À FAIRE

1. Sortir le code des notebooks vers des `.py` versionnés.
2. Figer l'environnement (`requirements.txt`).
3. Versionner les données (DVC).
4. Tracer les expériences (MLflow).

Google — Niveau 1 : automatisation du pipeline ML



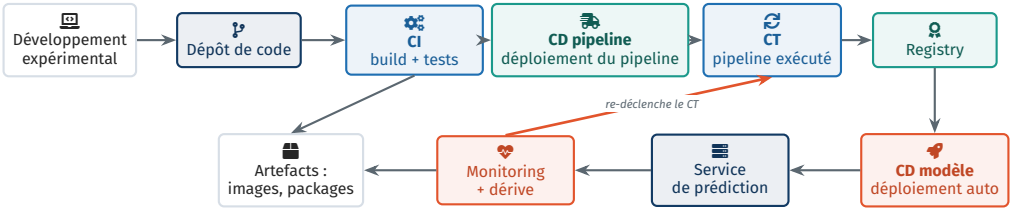
- Ce qui est livré n'est plus un modèle : c'est **le pipeline qui produit le modèle**.
- Le **Continuous Training** apparaît : ré-entraînement sur déclencheur.
- Deux étapes nouvelles et essentielles : **validation des données** et **validation du modèle**.
- Composants requis : orchestrateur de pipeline, *metadata store*, éventuellement *feature store*.

Niveau 1 : ce qu'il faut mettre en place

Composant	Rôle	Équivalent chez nous
Orchestrateur de pipeline	Enchaîner et reprendre les étapes	Jenkins (`Jenkinsfile.ml`)
Validation des données	Bloquer si schéma ou distribution anormaux	Deepchecks + Evidently
Validation du modèle	Comparer le candidat au champion	`register_best_model.py`
Metadata store	Historiser runs, params, métriques, artefacts	MLflow Tracking
Model Registry	Gouverner les versions et les stades	MLflow Registry (DagsHub)
Feature store	Partager les features train/serve	*Absent* — processeurs sérialisés
Déclencheur automatique	Lancer le CT sans humain	Détection de dérive Evidently

> Au niveau 1, le pipeline d'entraînement est automatisé… mais son **déploiement** reste manuel : on met à jour le service à la main. C'est précisément ce que le niveau 2 corrige.

Google — Niveau 2 : CI/CD complet du pipeline



Le niveau 2 ajoute deux boucles : le **CD du pipeline lui-même** (on livre une nouvelle version du pipeline) et le **CD du modèle** (le modèle promu part en production sans humain).

Niveau 2 : les étapes obligatoires du CI et du CD

Ce que doit tester la CI d'un projet ML

- Tests unitaires des fonctions de feature engineering.
- Test de convergence : l'entraînement produit-il bien un modèle ?
- Absence de `NaN` en sortie de preprocessing.
- Cohérence des schémas de données.
- Le modèle atteint-il un **seuil minimal** de performance ?
- Le modèle est-il meilleur que le champion actuel ?
- Tests d'intégration de l'API.

Ce que doit faire le CD

- Vérifier la compatibilité du modèle avec l'infrastructure cible.
- Vérifier que le service répond avant de basculer (*readiness*).
- Tester la latence sous charge attendue.
- Déployer de façon progressive (rolling, canary, blue/green).
- Permettre un **rollback immédiat**.
- Publier la version déployée comme métrique.

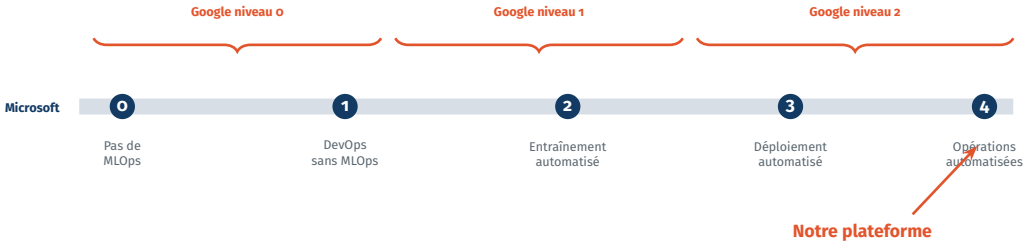
Quality gate : une étape qui bloque la chaîne si un critère n'est pas rempli. Sans *gates*, l'automatisation ne fait qu'accélérer la mise en production des erreurs.

Microsoft Azure — Une échelle en cinq niveaux

N	Nom	Description	Ce qui manque encore
0	Pas de MLOps	Notebooks, données non versionnées, livraison manuelle	Tout : versionnement, tests, automatisation
1	DevOps sans MLOps	Code versionné et testé, build automatisé, mais le ML reste manuel	Versionnement des données, tracking, CT
2	Entraînement automatisé	Pipeline d'entraînement reproductible, expériences tracées, registry en place	Déploiement automatique du modèle
3	Déploiement automatisé	Le modèle validé part en production automatiquement, tests A/B possibles	Monitoring complet et boucle de rétroaction
4	Opérations automatisées	Monitoring de bout en bout, réentraînement déclenché automatiquement, système auto-correctif	—

La grande différence avec Google : Microsoft insiste sur le **niveau 1**, celui où l'équipe fait *déjà* du bon génie logiciel mais n'a *pas encore* adapté ses pratiques aux données et aux modèles. C'est le niveau où stagnent la plupart des entreprises.

Correspondance entre les deux référentiels



Les deux échelles décrivent la même trajectoire : **automatiser d'abord l'entraînement**, **puis le déploiement**, **puis la décision de ré-entraîner**.

Grille d'évaluation — axe DONNÉES

Critère	Niveau 0	Niveau 1–2	Niveau 3	Niveau 4
Stockage	Fichier local, clé USB	Partage réseau	Stockage objet versionné	Data lake gouverné
Versionnement	Aucun (`data_v2_final.csv`)	Nommage conventionnel	**DVC / LakeFS**	Lignage automatique complet
Qualité	Contrôle visuel	Quelques `assert`	**Suites automatisées** (Deepchecks)	Contrats de données bloquants
Schéma	Implicite	Documenté	Validé en CI	Versionné et rétro-compatible
Dérive	Non mesurée	Vérifiée ponctuellement	**Rapport périodique**	**Déclencheur automatique**
Features	Recalculées ad hoc	Script partagé	Processeurs sérialisés	Feature store

Dans notre projet

Notre positionnement sur cet axe : **niveau 3–4**. DVC versionne données et artefacts, Deepchecks valide la qualité, Evidently mesure la dérive et **déclenche** le ré-entraînement. Il manque un *feature store* pour atteindre pleinement le niveau 4.

Grille d’évaluation — axe CODE & MODÈLE

Critère	Niveau 0	Niveau 1–2	Niveau 3	Niveau 4
Forme du code	Notebooks	Scripts `.py`	Modules testés et packagés	Bibliothèque interne réutilisable
Versionnement	Aucun ou local	Git	Git + revue de code	Git + conventions + *trunk based*
Environnement	« Ça marche chez moi »	`requirements.txt`	**Image Docker**	Image signée et scannée
Expériences	Notes manuelles	Fichier CSV de résultats	**MLflow Tracking**	Tracking + lignage complet
Sélection du modèle	À l’œil	Comparaison manuelle	**Automatisée sur métrique**	Automatisée + validation métier
Registry	Aucun	Dossier de `.pkl`	**MLflow Registry**	Registry + approbations et audit
Tests	Aucun	Quelques tests unitaires	**Unitaires + intégration**	+ tests de charge et de robustesse

Dans notre projet

Notre positionnement : **niveau 3**. Code modulaire, image Docker, MLflow Tracking et Registry, sélection automatique du meilleur modèle par ROC-AUC, tests Pytest en CI. Manquent : revue de code formalisée, tests de charge et signature d’images.

Grille d’évaluation — axe DÉPLOIEMENT & MONITORING

Critère	Niveau 0	Niveau 1–2	Niveau 3	Niveau 4
Mise en production	Copie manuelle	Script de déploiement	**Pipeline CD**	CD + promotion automatique
Stratégie	Arrêt / relance	Redémarrage planifié	**Rolling update**	Canary / blue-green / *shadow*
Rollback	Restauration manuelle	Sauvegarde de l’ancien binaire	`kubectl rollout undo`	Automatique sur seuil d’alerte
Métriques système	Aucune	Logs consultés à la demande	**Prometheus + Grafana**	SLO et budgets d’erreur
Métriques modèle	Aucune	Calculées hors ligne	**Exposées en temps réel**	Performance réelle vs *ground truth*
Alerting	Le client appelle	E-mail manuel	**Alertmanager**	Alertes + remédiation automatique
Ré-entraînement	Jamais	À la demande	Planifié	**Déclenché par la dérive**

Dans notre projet

Notre positionnement : **niveau 3–4**. Pipeline CD Jenkins vers Kubernetes, rolling update, Prometheus/Grafana/Alertmanager, ré-entraînement déclenché par la dérive. Manquent : SLO formalisés, canary et mesure de performance réelle (faute d’étiquettes en temps réel).

Auto-diagnostic : dix questions à se poser

1. Puis-je reproduire à l'identique le modèle en production ?
2. Sais-je sur quelles données exactes il a été entraîné ?
3. Combien de temps entre un commit et la production ?
4. Combien d'étapes manuelles dans cette chaîne ?
5. Que se passe-t-il si le modèle se dégrade cette nuit ?
6. Combien de temps pour revenir à la version précédente ?
7. Qui est alerté, et sur quel signal ?
8. Puis-je comparer deux modèles sur les mêmes données ?
9. Mon preprocessing d'entraînement est-il *le même* objet qu'en inférence ?
10. Si la personne qui a construit le pipeline part demain, que se passe-t-il ?

Lecture des résultats

0 à 3 réponses satisfaisantes → niveau 0–1 : commencez par le versionnement et le tracking. **4 à 7** → niveau 2–3 : automatisez le déploiement. **8 à 10** → niveau 4 : consolidez le monitoring et la gouvernance.

Où se situe notre plateforme — ce qui est en place

Capacité	Statut	Preuve dans le dépôt
Versionnement du code	✓	Dépôt Git, historique de commits
Versionnement des données	✓	`data.dvc`, `artifacts.dvc`, remote DagsHub
Tracking des expériences	✓	`mlflow.log_params` / `log_metrics` dans `train_model.py`
Model Registry	✓	`Fraud_Detection_Production`, alias `champion`
Champion / challenger	✓	`register_best_model.py` : promotion si `val_pr_auc` meilleure
Tests automatisés	✓	`test_api_mock.py`, `test_jenkins_scripts.py`, scénario $\geq$ 9/11
Validation des données	✓	`validate_data.py` (stage bloquant), `deepchecks.ipynb`
Conteneurisation	✓	Dockerfiles multi-stage backend et frontend
CI/CD automatisé	✓	`Jenkinsfile.ml`, `Jenkinsfile.deploy`
Déploiement orchestré	✓	`k8s/`, tags `<build>-<sha>`, rollback auto, `/live` + `/health`
Monitoring technique+métier	✓	13 métriques Prometheus, Pushgateway CI, 5 alertes, Grafana
Détection de dérive	✓	`drift_detection.py` (Evidently, seuil 30 %)
Ré-entraînement automatique	✓	`when { DRIFT_DETECTED == 'true' }`

Où se situe notre plateforme — ce qui manque

Capacité	Statut	Pourquoi, et ce qu'il faudrait
Feature store	✗	Les processeurs `.pkl` suffisent ici ; un feature store deviendrait nécessaire avec des agrégats temporels par carte.
Canary / A-B testing	✗	Rolling update seulement ; il faudrait un maillage de service (Istio) ou deux *Deployments* pondérés.
Performance réelle	✗	Pas d'étiquettes de fraude en temps réel ; il faudrait brancher le circuit des litiges bancaires.
SLO formalisés	✗	Les seuils d'alerte existent mais ne sont pas adossés à un budget d'erreur.

Verdict

La plateforme se situe au **niveau 2 de Google** et au **niveau 3–4 de Microsoft** : le pipeline d'entraînement *et* le déploiement sont automatisés, et le ré-entraînement est déclenché par un signal statistique. Les manques identifiés relèvent de la **sophistication**, pas des fondations.

Feuille de route : comment progresser d'un niveau

Transition	Actions prioritaires	Preuve d'aboutissement
0 → 1	Sortir des notebooks, Git, `requirements.txt`, tests unitaires	Le pipeline tourne sur une autre machine
1 → 2	DVC, MLflow, script d'entraînement paramétré, validation des données	Une expérience est reproductible à l'identique
2 → 3	Dockeriser, écrire le pipeline CI/CD, déployer sur un orchestrateur	Un commit atteint la production sans intervention
3 → 4	Monitoring ML, détection de dérive, déclencheur de CT, alerting	Le système se ré-entraîne seul et prévient les humains

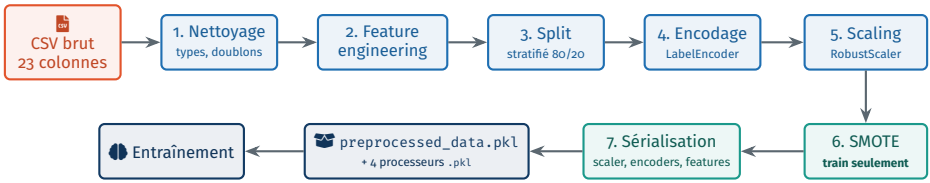
Trois erreurs classiques de montée en maturité

1. **Sauter des étapes** : déployer sur Kubernetes avant de savoir reproduire un entraînement.
2. **Automatiser sans garde-fous** : une CI sans *quality gates* livre les bugs plus vite.
3. **Outiller au lieu d'organiser** : installer MLflow ne crée pas la discipline de tracer.

5

Modélisation Machine Learning

Du CSV brut au jeu d'entraînement



L'ordre des étapes n'est pas négociable

Le **split intervient avant** l'encodage, le scaling et SMOTE. Inverser cet ordre provoque une **fuite de données** (*data leakage*) : le jeu de test influencerait les statistiques d'apprentissage, et les scores seraient artificiellement excellents… jusqu'à la production.

Étape 1 — Nettoyage des données

OPÉRATIONS RÉALISÉES

- Suppression de la colonne d'index `Unnamed: 0`.
- Conversion de `trans_date_trans_time` et `dob` en `datetime`.
- Vérification des doublons et des valeurs manquantes.
- Contrôle des bornes : montants négatifs, latitudes hors domaine.

CONSTAT SUR CE JEU

Aucune valeur manquante : c'est **inhabituel** et typique d'un dataset synthétique. En production réelle, prévoir une stratégie d'imputation **et** la sérialiser.

COLONNES ÉCARTÉES DE LA MODÉLISATION

- `first`, `last`, `street`, `city` — données personnelles.
- `trans_num`, `cc_num` — identifiants uniques, aucun pouvoir de généralisation.
- `unix_time` — redondant avec l'horodatage.
- `merchant` — très haute cardinalité, risque de sur-apprentissage.

> Écarter un identifiant unique n'est pas facultatif : un modèle peut **mémoriser** `trans_num` et afficher 100 % en test tout en étant inutile en production.

Étape 2 — Feature engineering temporel

```
df['trans_hour']      = ts.dt.hour
df['trans_day']       = ts.dt.day
df['trans_month']     = ts.dt.month
df['trans_year']      = ts.dt.year
df['trans_dayofweek'] = ts.dt.dayofweek
df['is_weekend']      = (df['trans_dayofweek'] >=
      5).astype(int)
df['day_period']      = df['trans_hour'].apply(_get_period)
```

POURQUOI CES VARIABLES

- Un horodatage brut est **inexploitable** par un arbre de décision.
- L'heure isole les transactions nocturnes, fortement corrélées à la fraude.
- Le découpage en périodes (*nuit*, *matin*, *après-midi*, *soirée*) réduit le bruit de la variable horaire.

`trans_year` est un **piège** : si le modèle apprend « année = 2019 », il ne généralisera pas en 2026. Cette variable doit être surveillée — c'est typiquement une source de dérive garantie.

Étape 2 — Feature géographique : la distance de Haversine

La distance orthodromique entre le domicile du porteur et le commerçant :

$$d = 2R \arcsin\sqrt{\sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)}$$

avec $R = 6371$ km, φ les latitudes et λ les longitudes en radians.

> Une transaction à 800 km du domicile, la nuit, sur du `misc_net` : trois signaux faibles qui, **combinés**, deviennent un signal fort.

```python
def _haversine_distance(lat1, lon1, lat2, lon2):
    R = 6371  # rayon terrestre en km
    lat1, lon1, lat2, lon2 = map(
        np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = (np.sin(dlat/2)**2
         + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2)
    return R * 2 * np.arcsin(np.sqrt(a))

df['distance_km'] = _haversine_distance(
    df['lat'], df['long'],
    df['merch_lat'], df['merch_long'])
```

Étape 2 — Âge et catégorisation du montant

```python
# Age du porteur AU MOMENT de la transaction (et non age courant)
df['age'] = (df['trans_date_trans_time'] - df['dob']).dt.days / 365.25

# Discretisation du montant en 4 tranches metier
df['amt_category'] = pd.cut(df['amt'],
                            bins=[0, 50, 100, 200, float('inf')],
                            labels=['faible', 'moyen', 'eleve', 'tres_eleve'])
```

LE DÉTAIL QUI COMPTE

L'âge est calculé **par rapport à la date de transaction**, pas par rapport à aujourd'hui. Sinon la même transaction historique donnerait un âge différent selon le jour où l'on relance le preprocessing : le résultat ne serait **pas reproductible**.

POURQUOI DISCRÉTISER LE MONTANT

- Rend le signal lisible pour le métier.
- Atténue l'effet des valeurs extrêmes (max observé : $28 949).
- Fournit une variable catégorielle complémentaire à `amt`.

Étapes 4 et 5 — Encodage et normalisation

ENCODAGE CATÉGORIEL

`LabelEncoder` par colonne (`category`, `gender`, `state`, `job`, `day_period`, `amt_category`).

- `fit_transform` sur le **train** uniquement.
- `transform` sur le test.
- Chaque encodeur est **conservé** dans `label_encoders.pkl`.

Acceptable ici car les modèles utilisés sont des arbres : ils ne supposent pas d'ordre entre les modalités. Pour une régression logistique, un *one-hot encoding* serait obligatoire.

NORMALISATION

`RobustScaler` plutôt que `StandardScaler` :

$$x' = \frac{x - \text{médiane}(x)}{Q_3(x) - Q_1(x)}$$

- La médiane et l'écart interquartile sont **insensibles aux valeurs extrêmes**.
- Or les montants de fraude *sont* des valeurs extrêmes : un `StandardScaler` les écraserait.
- Ajusté sur le train, appliqué au test, sauvegardé dans `scaler.pkl`.

Règle générale : **tout objet qui « apprend » quelque chose des données** (encodeur, scaler, imputeur) s'ajuste sur le train, jamais sur le test, et doit être sérialisé pour l'inférence.

Étape 6 — Traiter le déséquilibre des classes

Stratégie	Principe	Avantage	Risque
Sous-échantillonnage	Retirer des exemples majoritaires	Rapide	Perte d'information massive
Sur-échantillonnage naïf	Dupliquer les minoritaires	Simple	Sur-apprentissage par répétition
SMOTE	**Créer des exemples synthétiques par interpolation**	**Diversifie la classe rare**	Bruit si les classes se recouvrent
`class_weight`	Pondérer la fonction de coût	Aucune donnée créée	Réglage plus délicat
Seuil ajusté	Décaler le seuil de décision	Gratuit, réversible	Ne change pas l'apprentissage

Notre choix : SMOTE avec $k = 5$ voisins, appliqué **uniquement sur le jeu d'entraînement**, après scaling.

SMOTE sur l'ensemble *avant* le split est l'erreur la plus fréquente en détection de fraude : des voisins synthétiques se retrouvent des deux côtés du split.

SMOTE en pratique : les chiffres de notre pipeline

	Avant	Après
Train — légitimes	477 186	477 186
Train — fraudes	2 814	**477 186**
Total train	480 000	954 372
Ratio	1 : 170	1 : 1
Test — légitimes	119 296	119 296
Test — fraudes	704	704
Taux de fraude test	0,59 %	**0,59 %**

Le point essentiel

Le **jeu de test reste intact**, avec son déséquilibre naturel de 0,59 %. C'est la seule façon d'obtenir une estimation honnête de ce que fera le modèle en production.

474 372 exemples synthétiques ont été créés. Ils sont **utiles pour apprendre**, mais ils ne doivent jamais servir à **évaluer**.

Jeu utilisé pour la modélisation : `fraud_600k.csv` (600 000 transactions, 21 features après ingénierie).

Étape 7 — Sérialiser les processeurs : la parade au *skew*

```python
# --- A l'entrainement : on SAUVEGARDE tout ce qui a appris ---
joblib.dump(self.scaler,          'artifacts/preprocessing/scaler.pkl')
joblib.dump(self.label_encoders,  'artifacts/preprocessing/label_encoders.pkl')
joblib.dump(self.feature_names,   'artifacts/preprocessing/feature_names.pkl')
joblib.dump(self.smote_config,    'artifacts/preprocessing/smote_config.pkl')

# --- A l'inference : on RECHARGE les memes objets ---
class InferencePreprocessor(PreprocessingFraud):
    def __init__(self):
        super().__init__()
        self.load_processors()      # scaler + encoders + noms de features
```

Pourquoi c'est la ligne la plus importante du projet

L'API n'ré-implémente rien : elle **hérite de la même classe** `PreprocessingFraud` et recharge **les mêmes objets ajustés**. Le *training–serving skew* devient structurellement impossible — et le `Dockerfile` copie ces quatre `.pkl` dans l'image.

Les modèles candidats

Modèle	Principe	Force	Limite
Random Forest	Vote majoritaire d'arbres décorrélés (*bagging*)	Robuste, peu de réglages	Lent, moins précis ici
XGBoost	*Boosting* de gradient régularisé	Très performant, mature	Sensible aux hyperparamètres
LightGBM	**Boosting par croissance en feuilles, *histogram-based***	**Le plus rapide, excellent**	Sur-apprend sur petits jeux
CatBoost	Boosting avec traitement natif du catégoriel	Peu de préparation	Entraînement plus long

Pourquoi que des modèles d'arbres ?

Sur des **données tabulaires hétérogènes**, les ensembles d'arbres restent l'état de l'art : pas d'hypothèse de linéarité, insensibilité aux échelles, gestion naturelle des interactions, et entraînement en quelques secondes — ce qui rend le **ré-entraînement continu** économiquement viable.

Phase 1 — Résultats des modèles de référence

Modèle	ROC-AUC	Temps (s)	Commentaire
Random Forest	0,9926	103,89	Le plus lent, le moins précis
XGBoost	**0,9950**	8,07	Meilleur score de la phase
LightGBM	0,9942	**4,72**	**22 fois plus rapide** que RF
CatBoost	0,9931	11,22	Correct sans réglage

LECTURE

Les quatre modèles se tiennent en 0,24 point de ROC-AUC. Le **discriminant devient le temps d'entraînement**, car ce pipeline sera relancé automatiquement à chaque dérive.

> En MLOps, un modèle 20 fois plus rapide pour 0,1 point de moins est souvent le **meilleur choix opérationnel**.

Phase 2 — Optimisation par *RandomizedSearchCV*

PROTOCOLE

- 20 tirages aléatoires par modèle.
- Validation croisée 3 blocs.
- Métrique optimisée : ROC-AUC.
- 6 à 7 hyperparamètres par modèle.

Pourquoi *Randomized* et non *Grid* : à budget égal, la recherche aléatoire explore bien mieux un espace de grande dimension, car seuls quelques hyperparamètres comptent réellement.

Modèle	Base	Optimisé	Gain
Random Forest	0,9926	0,9949	+0,23 %
XGBoost	0,9950	0,9980	+0,30 %
LightGBM	0,9942	**0,9992**	**+0,50 %**
CatBoost	0,9931	0,9968	+0,37 %

Un signal à ne pas ignorer

Le meilleur score de validation croisée affiché pendant le réglage atteint **1,0000**. Sur un train équilibré par SMOTE, c'est **attendu mais trompeur** : les exemples synthétiques d'un bloc sont interpolés à partir de voisins présents dans les autres blocs. **Seul le score sur le jeu de test intact fait foi.**

Phase 3 — Apprentissage d'ensemble

DEUX STRATÉGIES TESTÉES

- **Stacking** : les 4 modèles optimisés servent de base, une régression logistique apprend à combiner leurs probabilités.
- **Voting (soft)** : moyenne des probabilités prédites, sans méta-modèle.

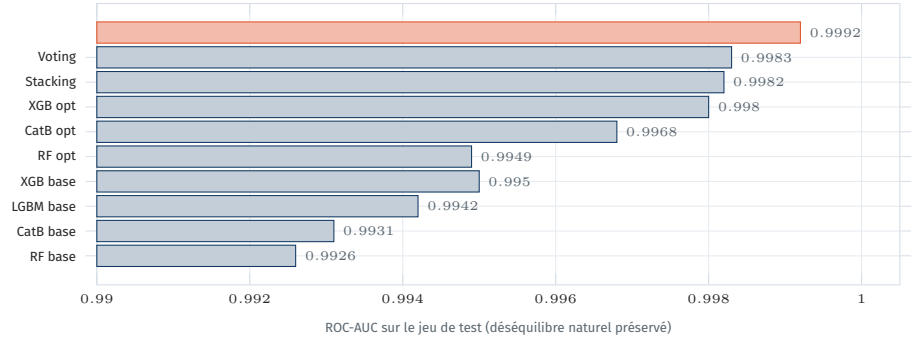
Ensemble	ROC-AUC	Temps (s)
Stacking	0,9982	1 713,7
Voting (soft)	0,9983	312,9
LightGBM opt.	**0,9992**	**11,1**

Conclusion contre-intuitive

Les ensembles **n'apportent rien** ici : ils sont **moins performants** qu'un LightGBM bien réglé, pour un coût d'entraînement **150 fois supérieur** et une complexité de déploiement bien plus grande (4 modèles à charger et versionner).

Décision : **LightGBM optimisé** part en production. La simplicité est une qualité opérationnelle.

Comparaison finale des dix modèles



Le modèle retenu est **LightGBM optimisé** : meilleur ROC-AUC *et* entraînement en 11 secondes.

Le modèle retenu et ses performances métier

Métrique (test)	Valeur
ROC-AUC	**0,9992**
Précision	0,9475
Rappel	0,8722
F1-score	0,9083

Sur 704 fraudes réelles du jeu de test : environ **614 détectées** et **90 manquées**, pour environ 34 fausses alertes.

Lecture critique

Un ROC-AUC de 0,9992 est **suspicieusement élevé** pour de la fraude réelle. Trois explications :

- données **synthétiques**, donc générées par des règles ;
- séparation temporelle absente (le split est aléatoire, pas chronologique) ;
- le rappel de 0,87 est, lui, un chiffre réaliste et plus informatif.

En production réelle, on utiliserait un **split temporel** : entraîner sur janvier–septembre, tester sur octobre–décembre.

Les pièges de modélisation à connaître

Piège	Manifestation	Parade
Fuite de données	Score parfait en test, échec en production	Split **avant** toute transformation
SMOTE avant split	CV à 1,0000	SMOTE sur le train uniquement
Fuite temporelle	Le modèle « voit le futur »	Split chronologique
Identifiants en features	Mémorisation	Écarter `trans_num`, `cc_num`
Accuracy comme critère	99,4 % sans rien apprendre	Rappel, précision, PR-AUC
Sur-optimisation du seuil	Seuil calé sur le jeu de test	Jeu de validation dédié
Modèle trop lourd	CT trop coûteux pour être automatisé	Arbitrer performance / temps

Les quatre premiers pièges produisent tous le **même symptôme** : d'excellents scores hors ligne suivis d'une déception en production. C'est la raison pour laquelle le monitoring est indispensable — il constitue le **seul juge de paix**.

6

Versionnement des données et du code

Pourquoi Git ne suffit pas

CE QUE GIT FAIT TRÈS BIEN

- Historiser des fichiers texte.
- Calculer des *diff*s ligne à ligne.
- Fusionner des contributions parallèles.
- Revenir à n'importe quel état passé.

CE QUE GIT FAIT TRÈS MAL

- Stocker un CSV de 500 Mo (chaque version est dupliquée intégralement).
- Diffuser un modèle sérialisé de 2 Go.
- Comparer deux fichiers binaires.

Les limites concrètes

- GitHub : avertissement au-delà de **50 Mo** par fichier, refus à **100 Mo**.
- Un dépôt qui a contenu un gros binaire reste lourd **pour toujours** (l'objet reste dans l'historique).
- Un `git clone` devient inutilisable.

La solution : dissocier

Git versionne les **métadonnées** (un pointeur de quelques octets), un **stockage objet** héberge les données. C'est exactement le principe de **DVC**.

Discipline Git pour un projet ML

ORGANISATION DES BRANCHES

- `main` — toujours déployable.
- `develop` — intégration.
- `feature/*` — une fonctionnalité.
- `experiment/*` — une piste de modélisation.
- `hotfix/*` — correction urgente.

MESSAGES DE COMMIT

`feat:`, `fix:`, `data:`, `model:`, `ci:`, `docs:`, `test:` — la convention permet de générer automatiquement un journal des changements.

CE QUI VA DANS GIT

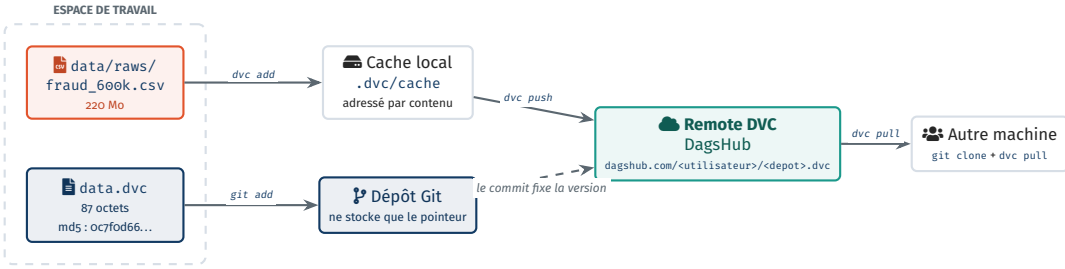
✅ scripts `.py`, notebooks nettoyés, `Dockerfile`, `Jenkinsfile`, manifestes `k8s/`, `requirements.txt`, fichiers `.dvc`, tests.

CE QUI NE VA JAMAIS DANS GIT

❌ datasets, modèles `.pkl`, `mlruns/`, `venv/`, `node_modules/`, `__pycache__/`, sorties de notebooks volumineuses, **et surtout aucun secret** (jetons, mots de passe, `kubeconfig`).

> Un secret poussé une fois est **compromis définitivement** : il faut le révoquer, pas seulement le supprimer.

DVC — le principe en une image



Le lien est le **hash MD5** : un commit Git désigne un pointeur, qui désigne un contenu unique. Revenir à un commit d'il y a trois mois puis lancer `dvc pull` restitue **exactement** les données de l'époque.

Anatomie d'un fichier `.dvc`

`data.dvc` — 87 octets dans Git :

```
outs:
- md5: 0c7f0d668c0b2266d2bd1c2de6a84e45.dir
  nfiles: 3
  hash: md5
  path: data
```

`artifacts.dvc` :

```
outs:
- md5: 2f0f65e351d5093ff5d6655df1367da4.dir
  size: 2297287435      # 2,3 Go
  nfiles: 44
  hash: md5
  path: artifacts
```

LECTURE

- Le suffixe `.dir` indique un **répertoire entier** versionné comme un tout.
- `nfiles` et `size` servent aux contrôles d'intégrité.
- **2,3 Go d'artefacts** et **44 fichiers** suivis — impensable dans Git.

> **Deux pointeurs seulement** dans le dépôt (`data.dvc` et `artifacts.dvc`) suffisent à figer l'intégralité des données et des modèles du projet.

Les commandes DVC à connaître

```
# --- Initialisation (une seule fois) ---
dvc init                                  # cree .dvc/ et l'ajoute a Git
dvc remote add -d origin https://dagshub.com/<utilisateur>/<depot>.dvc

# --- Versionner des donnees ---
dvc add data/                             # cree data.dvc + met en cache
git add data.dvc .gitignore && git commit -m "data: dataset v1"
dvc push                                  # envoie le contenu vers le remote

# --- Recuperer (sur une autre machine, ou dans Jenkins) ---
git clone <repo> && cd repo
dvc pull                                  # tout
dvc pull data/raws/fraud_600k.csv --jobs 1   # un seul fichier

# --- Naviguer dans les versions ---
git checkout <commit> && dvc checkout     # restaure les donnees de ce commit
dvc status                                # ecart entre cache, remote et espace de travail
dvc diff HEAD~1                           # ce qui a change entre deux versions
```

Règle mnémotechnique : **`git checkout`** change le code, **`dvc checkout`** change les données. Les deux vont toujours ensemble.

Le remote : DagsHub

POURQUOI DAGSHUB

- Un seul service pour **Git + DVC + MLflow**.
- Remote DVC et serveur MLflow fournis, sans infrastructure à gérer.
- Visualisation des *diffs* de données et des expériences.
- Authentification par jeton unique.

LES TROIS URL DU PROJET

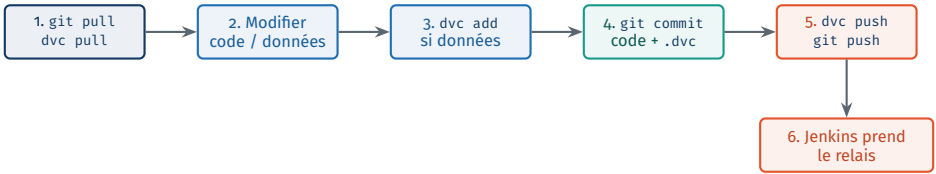
`.../<depot>` — dépôt Git
`.../<depot>.dvc` — remote DVC
`.../<depot>.mlflow` — serveur MLflow

```
# Configuration LOCALE, non versionnee (--local
# ecrit dans .dvc/config.local, ignore par Git)
dvc remote modify origin --local \
    user <utilisateur>
dvc remote modify origin --local \
    password $DAGSHUB_TOKEN
```

Le piège à éviter absolument

Sans l'option `–local`, les identifiants sont écrits dans `.dvc/config`, qui **est versionné** : le jeton part dans Git et se retrouve dans tous les clones. Le jeton doit venir d'une **variable d'environnement** ou du gestionnaire de secrets de la CI.

Le cycle de travail quotidien



ORDRE À RESPECTER

`dvc push` **avant** `git push`. Dans le cas contraire, un collègue récupère un pointeur dont le contenu n'existe pas encore sur le remote — et `dvc pull` échoue.

Dans notre pipeline Jenkins, l'étape *Setup Environment* fait précisément ces `dvc pull` ciblés, fichier par fichier, avec `–jobs 1` pour limiter la charge réseau.

Aller plus loin : les pipelines DVC

```yaml
# dvc.yaml
stages:
  preprocess:
    cmd: python src/preprocessing.py
    deps:
      - data/raws/fraud_600k.csv
      - src/preprocessing.py
    outs:
      - artifacts/preprocessing/preprocessed_data.pkl
      - artifacts/preprocessing/scaler.pkl
  train:
    cmd: python src/train.py
    deps:
      - artifacts/preprocessing/preprocessed_data.pkl
      - src/train.py
    params:
      - n_estimators
      - learning_rate
    outs:
      - models/model.pkl
    metrics:
      - metrics.json:
          cache: false
```

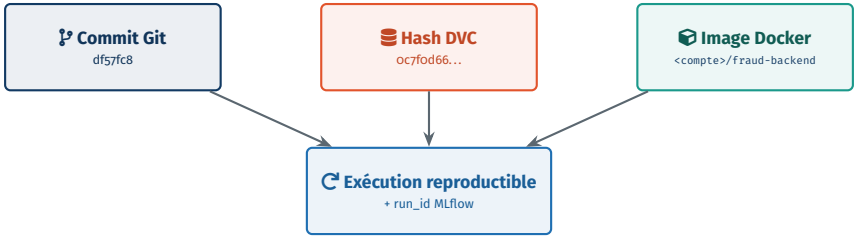
CE QUE CELA APPORTE

- `dvc repro` ne rejoue **que les étapes dont les dépendances ont changé**.
- `dvc dag` affiche le graphe du pipeline.
- `dvc metrics diff` compare les métriques entre deux commits.
- `dvc exp run` lance des expériences paramétrées.

Dans notre projet

Notre projet utilise DVC pour le **stockage versionné** et confie l'**orchestration** à Jenkins. Un `dvc.yaml` serait le prochain gain : la reproductibilité passerait d'une convention à une garantie.

Reproductibilité : le triplet indissociable



Test de reproductibilité

Sur une machine vierge : `git clone`, `git checkout <sha>`, `dvc pull`, `docker build`. Si le résultat diffère, **quelque chose n'est pas versionné**.

Les causes fréquentes d'écart

`random_state` non fixé, version de librairie non épinglée, chemin absolu codé en dur, dépendance à l'heure système, parallélisme non déterministe.

Comparaison des solutions de versionnement de données

Outil	Principe	Points forts	Limites
DVC	Pointeurs Git + stockage objet	Léger, agnostique, pipelines inclus	Pas de requêtes sur les données
Git LFS	Substitution de gros fichiers dans Git	Transparent pour l'utilisateur	Quotas coûteux, pas de pipelines
LakeFS	Branches sur un data lake objet	*Branch*/*merge* à l'échelle du lac	Infrastructure lourde
Delta Lake	Journal de transactions sur Parquet	ACID, *time travel* SQL	Écosystème Spark
Pachyderm	Pipelines versionnés sur K8s	Lignage automatique	Complexité opérationnelle

Pour un projet de cette taille, **DVC est le bon compromis** : il s'installe en une commande, s'utilise avec des réflexes Git déjà acquis, et n'impose aucune plateforme.

Bonnes pratiques et pièges du versionnement

À faire

- Committer le `.dvc` **en même temps** que le code qui l'utilise.
- Épingler les versions dans `requirements.txt`.
- Étiqueter les jalons : `git tag data-v2`.
- Documenter la provenance des données.
- Nettoyer les sorties de notebooks avant commit.
- Utiliser `dvc remote modify --local` pour les identifiants.

À ne pas faire

- Nommer les versions `data_final_v3_ok.csv`.
- `git push` sans `dvc push`.
- Écrire un jeton dans un fichier versionné.
- Laisser `mlruns/` ou `artifacts/` entrer dans Git.
- Modifier un fichier suivi par DVC sans `dvc add`.
- Considérer qu'un notebook est un livrable de production.

Vérification utile avant chaque *push* : `git status` **et** `dvc status` — les deux doivent être propres.

7

MLflow : tracking et gouvernance des modèles

Le problème que MLflow résout

LA SITUATION SANS OUTIL

Dix modèles, quatre jeux d'hyperparamètres, trois versions de données : **120 combinaisons**. Les résultats sont dans…

- la sortie console d'un notebook fermé depuis ;
- un tableur `resultats_v2.xlsx` ;
- la mémoire de la personne qui a lancé le calcul.

LES QUESTIONS SANS RÉPONSE

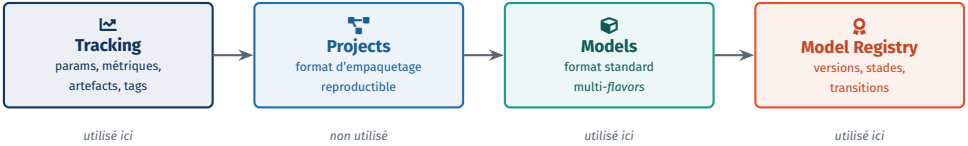
« Quel modèle est en production ? » — « Avec quels hyperparamètres ? » — « Sur quelles données ? » — « Peut-on revenir à celui de la semaine dernière ? »

Ce qu'apporte MLflow

- Un **journal automatique** de chaque exécution : paramètres, métriques, artefacts, code, environnement.
- Une **interface de comparaison** entre runs.
- Un **registre** pour gouverner les versions de modèles.
- Une **API** pour interroger tout cela depuis un script — donc depuis un pipeline CI/CD.

Le dernier point est décisif : c'est ce qui rend la **sélection automatique du meilleur modèle** possible.

Les quatre composants de MLflow



- **Tracking** — le journal des expériences. C'est le point d'entrée de tout projet.
- **Projects** — un `MLproject` décrit commande et environnement ; nous préférons Docker pour ce rôle.
- **Models** — un modèle sauvegardé avec ses métadonnées, chargeable indifféremment en Python, en R ou via une API REST.
- **Registry** — le référentiel qui répond à la question « **quel modèle est en production ?** ».

Le vocabulaire du tracking

Notion	Définition	Exemple dans notre projet
Experiment	Un regroupement logique de runs	`continuous_training_pipeline`
Run	Une exécution d'entraînement	`LightGBM_CT_20260906_143012`
Parameter	Une entrée fixée avant l'exécution	`n_estimators=100`, `model_name`
Metric	Une mesure de sortie (historisable)	`roc_auc`, `f1_score`, `training_time_seconds`
Artifact	Un fichier produit par le run	`model/model.pkl`, courbe ROC
Tag	Une annotation libre	`stage=continuous_training`
Run ID	L'identifiant unique et immuable	`a3f9c1e8...`

La distinction **paramètre / métrique** n'est pas cosmétique : un paramètre est une valeur unique, une métrique peut être enregistrée à chaque *epoch* et donne une courbe. On **filtre** sur les paramètres, on **trie** sur les métriques.

Le tracking dans notre script d'entraînement

```
mlflow.set_experiment("continuous_training_pipeline")   # URI DagsHub : variable d'env

for name, model in candidate_models().items():
    with mlflow.start_run(run_name=f"{name}_CT_{run_timestamp}"):
        mlflow.set_tags({'git_commit': os.getenv('GIT_COMMIT'),   # commit du code
                         'dvc_data_md5': md5,                     # hash DVC des donnees
                         'ct_batch': ct_batch, 'model_name': name})
        mlflow.log_params(model.get_params())          # tous les hyperparametres

        model.fit(X_train, y_train)                    # train reechantillonne par SMOTE
        proba_val = model.predict_proba(X_val)[:, 1]  # VALIDATION reelle : jamais SMOTE
        mlflow.log_metrics({f'val_{k}': v for k, v in compute_metrics(y_val, proba_val).items()})
        threshold, _ = find_best_threshold(y_val, proba_val)      # seuil F1-optimal
        mlflow.log_metric('val_best_threshold', threshold)
        # test : rapport seulement, jamais lu pour la selection
        mlflow.log_metrics({f'test_{k}': v for k, v in test_metrics.items()})

        mlflow.sklearn.log_model(model, "model")       # le modele devient un artefact
```

Le `with mlflow.start_run()` garantit la clôture du run **même en cas d'exception**. Les tags `git_commit` et `dvc_*_md5` relient chaque run au **code** et aux **données** exacts qui l'ont produit.

Configuration du serveur de tracking

```
# --- Option 1 : local, zero configuration ---
#   les runs vont dans ./mlruns
mlflow ui --port 5000

# --- Option 2 : serveur avec base de donnees ---
mlflow server \
  --backend-store-uri postgresql://... \
  --default-artifact-root s3://mon-bucket/mlflow \
  --host 0.0.0.0 --port 5000

# --- Option 3 : notre choix, DagsHub ---
export MLFLOW_TRACKING_URI=\
  https://dagshub.com/<utilisateur>/<depot>.mlflow
export MLFLOW_TRACKING_USERNAME=<utilisateur>
export MLFLOW_TRACKING_PASSWORD=$DAGSHUB_TOKEN
```

DEUX MAGASINS DISTINCTS

- **Backend store** — les métadonnées (params, métriques) : fichiers ou base SQL.
- **Artifact store** — les fichiers volumineux : disque, S3, GCS.

Le mode `./mlruns` local ne se partage pas et ne survit pas à un conteneur éphémère. Dès qu'un pipeline CI entraîne des modèles, un **serveur distant devient obligatoire**.

Dans notre projet

Les trois variables d'environnement sont injectées par Jenkins depuis le *credential* `dagshub-token`.

Interroger MLflow depuis un script : la clé de l'automatisation

```python
experiment = mlflow.get_experiment_by_name("continuous_training_pipeline")

df = mlflow.search_runs(                    # renvoie un DataFrame pandas
    experiment_ids=[experiment.experiment_id],
    filter_string=f"metrics.val_pr_auc > 0 and tags.ct_batch = '{ct_batch}'",
    order_by=["metrics.val_pr_auc DESC"],   # metrique de VALIDATION
    max_results=1                           # le meilleur du lot : le challenger
)

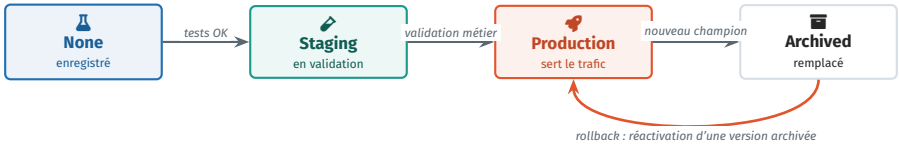
challenger = df.iloc[0]
run_id     = challenger.run_id
score      = challenger['metrics.val_pr_auc']
```

POURQUOI C'EST STRUCTURANT

La sélection du meilleur modèle n'est plus un **jugement humain** sur une capture d'écran : c'est une **requête déterministe** qu'un pipeline peut exécuter à 3 h du matin.

Le filtre `tags.ct_batch` limite la compétition aux modèles **de ce lot** : sinon un vieux run excellent serait réélu à chaque exécution. La marge `MIN_IMPROVEMENT` évite de promouvoir du bruit statistique.

Le Model Registry : cycle de vie d'une version



CE QUE GARANTIT LE REGISTRE

- Une version est **immuable** : on n'écrase jamais un modèle.
- Un seul modèle porte l'alias `champion` (ou le stade `Production`) à un instant donné.
- Chaque version conserve le lien vers son `run_id` d'origine.

Le rollback devient trivial : il suffit de repromouvoir la version précédente. **Aucun ré-entraînement**. Depuis MLflow 2.9, les stades sont dépréciés au profit des **alias** (`@champion`), que notre script utilise en priorité.

La promotion automatique : champion contre challenger

```python
# jenkins/register_best_model.py
def decide_promotion(challenger_id, challenger_score, champion_id, champion_score,
                     min_improvement=0.0):
    if champion_id is None:          return True,  "aucun champion enregistre"
    if challenger_id == champion_id: return False, "deja champion"
    if challenger_score > champion_score + min_improvement:
        return True, "le challenger bat le champion"
    return False, "le champion reste en place"

champion = client.get_model_version_by_alias("Fraud_Detection_Production", "champion")
promoted, reason = decide_promotion(run_id, score, champion.run_id, champion_score)
if promoted:
    version = mlflow.register_model(f"runs:/{run_id}/model", "Fraud_Detection_Production")
    client.set_registered_model_alias("Fraud_Detection_Production", "champion", version.version)
    export_model(client, version)    # model.pkl + production.pkl + metadata.json
write_promotion(promoted, reason)    # promotion.txt, lu par Jenkins (readFile)
```

LE CONTENU DE METADATA.JSON

`run_id`, algorithme, version du registre, métriques `val_*` et `test_*`, et surtout `threshold` : le **seuil de décision** optimisé sur la validation, que l'API applique.

Le pipeline de déploiement relance ce script avec `--export-champion` : l'image embarque **le modèle du registre**, pas un fichier local. La règle de décision est testée unitairement.

Servir un modèle : deux stratégies

	Chargement depuis MLflow	Modèle embarqué dans l'image
Mise en œuvre	`mlflow.pyfunc.load_model("models:/.../Production")`	`COPY` du `.pkl` dans le Dockerfile
Mise à jour	Redémarrage du pod suffit	Reconstruction de l'image
Dépendance	Le serveur MLflow devient critique	Aucune dépendance externe
Démarrage	Téléchargement à chaque démarrage	Immédiat
Traçabilité	Native	Via `metadata.json`
Reproductibilité	L'image ne suffit pas	**L'image est autoportante**

Dans notre projet

Notre choix : le modèle embarqué. Une image Docker devient un livrable **complet et immuable** : le pod démarre sans réseau vers DagsHub, et l'image `:<build>-<sha>` *est* la version du modèle. Le prix à payer : une reconstruction d'image à chaque promotion — ce que le pipeline Jenkins fait de toute façon.

Le format MLflow Model et la signature

```
# fichier MLmodel genere automatiquement
artifact_path: model
flavors:
  python_function:
    loader_module: mlflow.sklearn
    python_version: 3.12.1
  sklearn:
    pickled_model: model.pkl
    sklearn_version: 1.6.1
signature:
  inputs: '[{"name": "amt", "type": "double"},
            {"name": "distance_km", "type": "double"},
            {"name": "category", "type": "long"}]'
  outputs: '[{"type": "long"}]'
run_id: a3f9c1e8b24d4f7a...
```

LES *FLAVORS*

Un même modèle est décrit sous plusieurs formes : `sklearn` pour un chargement natif, `python_function` pour une interface universelle `predict(DataFrame)`.

LA SIGNATURE

Elle déclare le **schéma d'entrée attendu**. MLflow peut alors **refuser** une requête mal formée au lieu de produire une prédiction silencieusement fausse.

> Loguer une signature (`signature=infer_signature(X, y)`) est une des meilleures protections contre le *training–serving skew*.

Ce que l'on regarde dans l'interface MLflow

VUE « EXPÉRIENCE »

- Tableau de tous les runs, triable par métrique.
- Colonnes de paramètres pour repérer les configurations gagnantes.
- Filtres : `params.model_name = 'LightGBM'`.

VUE « COMPARAISON »

- Sélection de plusieurs runs côte à côte.
- Graphe de coordonnées parallèles : quel hyperparamètre influence réellement la métrique.
- Courbes de métriques superposées.

VUE « RUN »

- Tous les paramètres et métriques du run.
- Artefacts téléchargeables (modèle, figures, rapports).
- Commit Git source, si l'exécution vient d'un dépôt propre.

VUE « REGISTRY »

- Liste des modèles nommés et de leurs versions.
- Stade courant de chaque version.
- Historique des transitions, avec auteur et horodatage.

L'interface sert à **comprendre** ; l'API sert à **automatiser**. Un projet mature utilise les deux.

Bonnes pratiques MLflow

À faire

- Nommer les runs de façon parlante et horodatée.
- Loguer **aussi** la version des données (hash DVC) comme paramètre.
- Loguer le temps d'entraînement : c'est une contrainte opérationnelle.
- Loguer les figures (ROC, matrice de confusion) comme artefacts.
- Déclarer une *signature* de modèle.
- Utiliser un *experiment* par objectif métier.

À éviter

- Tout envoyer dans l'expérience `Default`.
- Loguer des données sensibles en clair dans les paramètres.
- Loguer un modèle de plusieurs Go à chaque itération.
- Se fier au dernier run plutôt qu'au meilleur.
- Promouvoir en `Production` sans test d'intégration.
- Confondre *tracking* et *registry* : l'un journalise, l'autre gouverne.

`mlflow.autolog()` loge automatiquement paramètres, métriques et modèle pour la plupart des bibliothèques — pratique pour démarrer, insuffisant pour un pipeline de production où l'on veut **choisir** ce qui est tracé.

8

API et application Web

Pourquoi exposer le modèle par une API

LE BESOIN

Un `.pkl` n'est utilisable que par un script Python disposant des mêmes versions de librairies. Une API HTTP, elle, est consommable par **n'importe quel client** : une application mobile, un système bancaire en Java, un tableur, un autre microservice.

CE QUE L'API ENCAPSULE

1. Le chargement du modèle et des processeurs.
2. Le preprocessing d'inférence.
3. La prédiction.
4. L'instrumentation (métriques, journaux).
5. La gestion des erreurs.

Le contrat d'interface

L'API définit un **contrat** : un schéma d'entrée, un schéma de sortie, des codes d'erreur. Tant que le contrat est respecté, le modèle peut être remplacé, réentraîné ou changé d'algorithme **sans qu'aucun client ne change une ligne de code**.

C'est ce découplage qui permet le déploiement continu de modèles : le frontend ne sait pas — et n'a pas besoin de savoir — qu'un LightGBM a remplacé un XGBoost cette nuit.

Pourquoi FastAPI

	FastAPI	Flask	Django REST
Modèle d'exécution	ASGI, asynchrone natif	WSGI, synchrone	WSGI
Validation	**Pydantic, automatique**	Manuelle	Serializers
Documentation	**OpenAPI / Swagger générée**	Extension requise	Extension requise
Typage	Annotations Python exploitées	Aucun	Partiel
Performance	Élevée (Starlette + Uvicorn)	Moyenne	Moyenne
Courbe d'apprentissage	Douce	Très douce	Raide

L'ARGUMENT DÉCISIF POUR LE ML

La **validation automatique** : un champ manquant ou un type incorrect produit une erreur `422` explicite **avant** d'atteindre le modèle. Sans cela, une chaîne envoyée à la place d'un nombre se transforme en prédiction silencieusement absurde.

Le `/docs` Swagger généré automatiquement sert de **documentation vivante** et de console de test pour les équipes métier.

Les endpoints de notre API

INFÉRENCE ET MÉTADONNÉES

Verbe	Chemin	Rôle
GET	`/`	Message d'accueil
GET	`/features`	Features attendues
GET	`/model-info`	Métadonnées du modèle actif
GET	`/model-metadata`	run_id, métriques, **seuil**
POST	`/predict`	Classe **et** probabilité, unitaire ou lot
POST	`/predictCSV`	Batch CSV → CSV annoté

OBSERVABILITÉ ET CI/CD

Verbe	Chemin	Consommateur
GET	`/live`	*livenessProbe*
GET	`/health`	*readinessProbe* (503)
GET	`/metrics`	Prometheus
GET	`/metrics-summary`	Page monitoring
GET	`/drift-status`	Frontend, alerting
GET	`/drift-report`	Frontend (HTML Evidently)
POST	`/update-drift-metrics`	**Jenkins**
POST	`/upload-drift-report`	**Jenkins**

Les deux `POST` Jenkins exigent l'en-tête `X-API-Token` dès que `API_ADMIN_TOKEN` est définie (Secret Kubernetes) : sinon, n'importe qui pourrait **falsifier le score de dérive**. Les résultats de CI passent désormais par le *Pushgateway*, plus par l'API.

Le chargement au démarrage

```python
model = None; preprocessor = None; decision_threshold = 0.5   # etat global du service

@app.on_event("startup")
async def startup_event():
    global model, preprocessor, model_metadata, decision_threshold
    # 1. Les processeurs ajustes a l'entrainement (scaler, encoders, features)
    preprocessor = InferencePreprocessor()      # RuntimeError si un artefact manque
    # 2. Le champion exporte du registre MLflow, embarque dans l'image
    model_dir = Path(os.getenv('MODEL_REGISTRY_DIR', str(DEFAULT_MODEL_DIR)))
    model = pickle.load(open(model_dir / "model.pkl", "rb"))
    metadata = json.loads((model_dir / "metadata.json").read_text())
    # 3. Le seuil : FRAUD_THRESHOLD > metadata["threshold"] (optimise en validation) > 0.5
    decision_threshold = resolve_threshold(metadata)
    MODEL_LOADED_STATUS.set(1); DECISION_THRESHOLD.set(decision_threshold)
```

POURQUOI AU DÉMARRAGE

Charger le modèle à chaque requête coûterait des centaines de millisecondes. Ici, le coût est payé **une fois**, à l'initialisation du processus.

Corollaire : le pod ne doit **pas** recevoir de trafic avant la fin du chargement. C'est exactement le rôle de la ***readinessProbe*** sur `/health`, qui renvoie `503` tant que le modèle n'est pas chargé.

Le contrat d'entrée avec Pydantic

```python
class TransactionInput(BaseModel):
    trans_date_trans_time: str = "2025-01-01 12:00:00"
    amt:         float = 100.0
    lat:         float = 40.7128
    long:        float = -74.0060
    merch_lat:   float = 40.7200
    merch_long:  float = -74.0100
    category:    str   = "grocery_pos"
    gender:      str   = "M"
    state:       str   = "NY"
    job:         str   = "Developer"
    dob:         str   = "1990-01-01"
```

Onze champs seulement : ce sont les **données brutes** de la transaction. Les 21 features du modèle en sont **dérivées** par le préprocesseur.

CE QUE PYDANTIC APPORTE GRATUITEMENT

- Conversion de types (`"100"` → `100.0`).
- Erreur `422` détaillée si un champ manque.
- Génération du schéma OpenAPI.
- Valeurs par défaut qui remplissent l'exemple Swagger.

Amélioration possible : contraindre les domaines (`amt: float = Field(gt=0)`, `gender: Literal['M','F']`). Le modèle est alors protégé des entrées absurdes **par construction**.

L'endpoint `/predict`

```python
def _predict(df_input):
    X = preprocessor.preprocess_inference(df_input)          # MEME preprocessing
    proba = model.predict_proba(X)[:, 1]
    return (proba >= decision_threshold).astype(int), proba  # seuil EXPLICITE

@app.post("/predict")
async def predict(request: Union[TransactionInput, List[TransactionInput]]):
    if model is None or preprocessor is None:
        raise HTTPException(status_code=503, detail="Model or preprocessor not available")
    start_time = time.time()
    try:
        items = request if isinstance(request, list) else [request]
        predictions, proba = _predict(pd.DataFrame([i.model_dump() for i in items]))
    except Exception as e:
        PREDICTION_ERRORS.inc()
        raise HTTPException(status_code=500, detail=f"Prediction error: {e}")
    _record_predictions(predictions)        # compteurs + taux sur les 1000 dernieres
    PREDICTION_LATENCY.observe(time.time() - start_time)
    return {"predictions": predictions.tolist(), "probabilities": proba.tolist(),
            "threshold": decision_threshold, "count": len(predictions)}
```

`Union[T, List[T]]` sert **une** transaction ou **un lot** sur le même chemin. Le `503` distingue « pas prêt » de « erreur de traitement ». Le seuil n'est plus le 0,5 implicite de `model.predict` : c'est celui **optimisé sur la validation**, renvoyé avec la probabilité.

Sondes de vie : `/live`, `/health` et les codes de retour

Situation	Réponse	Conséquence Kubernetes
Modèle chargé	`/health` → `200 {"status":"ok"}`	Le pod reçoit du trafic
Chargement en cours	`/health` → `503` `{"status":"degraded"}`	*readinessProbe* échoue : **pas de trafic**
Processus figé	`/live` ne répond plus (timeout)	*livenessProbe* échoue → redémarrage
Erreur d'inférence	`500` sur `/predict`	Compteur d'erreurs, alerte

Défaut corrigé dans cette version

Avant : `/health` renvoyait toujours `200`, même en état `degraded` : Kubernetes envoyait du trafic à un pod qui répondait `503` sur `/predict`. **Après** : `/health` renvoie **`503`** tant que le modèle n'est pas chargé, et la *livenessProbe* interroge `/live`, qui répond toujours : un modèle absent **retire le pod du trafic** sans boucle de redémarrages.

Instrumenter l'API pour Prometheus

```python
from prometheus_client import Counter, Histogram, Gauge, make_asgi_app

PREDICTIONS_TOTAL   = Counter('fraud_predictions_total',
                              'Total des predictions par resultat', ['result'])
FRAUD_RATE          = Gauge('fraud_rate_current',     # fenetre glissante (deque)
                            'Taux de fraude sur les 1000 dernieres predictions (0-1)')
PREDICTION_LATENCY  = Histogram('fraud_prediction_latency_seconds',
                                'Latence des predictions unitaires (/predict)',
                                buckets=[0.05, 0.1, 0.25, 0.5, 1.0, 2.0, 5.0])
MODEL_LOADED_STATUS = Gauge('fraud_model_loaded', 'Modele charge : 1=oui, 0=non')
DECISION_THRESHOLD  = Gauge('fraud_decision_threshold', 'Seuil de decision applique')

app.mount("/metrics", make_asgi_app())   # expose /metrics au format OpenMetrics
```

CHOISIR LE BON TYPE

- **Counter** — ne fait que croître (volumes, erreurs).
- **Gauge** — monte et descend (taux, score de dérive).
- **Histogram** — distribution, permet les quantiles.

Les *buckets* de l'histogramme doivent encadrer la latence **attendue**. Ici de 50 ms à 5 s : c'est ce qui permet de calculer un `histogram_quantile(0.99, ...)` exploitable pour l'alerte « P99 > 2 s ».

Deux subtilités de déploiement derrière un Ingress

```python
app = FastAPI(
    title="Fraud Detection API",
    version="2.0.0",
    root_path="/api"        # <-- l'API est servie sous /api
)
cors_origins = os.getenv("CORS_ORIGINS",
    "http://localhost:3000,http://127.0.0.1:3000,http://fraud.local")
app.add_middleware(
    CORSMiddleware,
    allow_origins=[o.strip() for o in cors_origins.split(",")],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

LE PARAMÈTRE ROOT_PATH

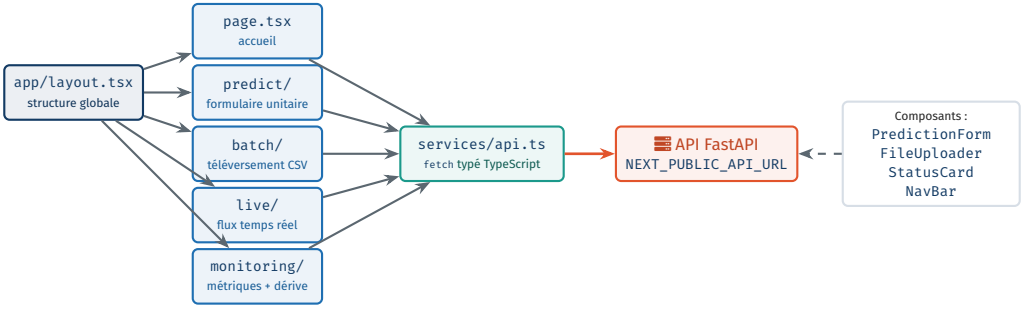
Quand un *reverse proxy* préfixe les chemins, l'application doit le savoir — sinon le Swagger généré pointe vers de mauvaises URL. C'est un problème **d'affichage de documentation**, pas de routage.

CORS

Un navigateur refuse par défaut qu'une page de `fraud.local` appelle `api.fraud.local`. La liste blanche d'origines est **indispensable** dès que frontend et API n'ont pas la même origine.

> `allow_origins=["*"]` avec `allow_credentials=True` est refusé par les navigateurs *et* dangereux. Toujours énumérer les origines.

L'application Web Next.js



- **App Router** de Next.js 14 : un dossier = une route, un `layout.tsx` partagé.
- **TypeScript** : les interfaces `PredictRequest` et `PredictionResponse` **documentent le contrat** de l'API côté client.
- **output : 'standalone'** dans `next.config.js` : produit un serveur Node minimal, ce qui allège fortement l'image Docker.

Le piège de la variable d'environnement du frontend

```
// services/api.ts
const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000';
```

```
# frontend/Dockerfile -- la valeur est figee AU MOMENT DU BUILD
ARG NEXT_PUBLIC_API_URL=http://localhost:8000
ENV NEXT_PUBLIC_API_URL=$NEXT_PUBLIC_API_URL
RUN npm run build
```

```
# Jenkinsfile.deploy -- d'ou l'argument passe au build
docker build --build-arg NEXT_PUBLIC_API_URL=http://api.fraud.local \
             -t ${IMAGE_FRONTEND}:${IMAGE_TAG} -f frontend/Dockerfile .
```

Pourquoi c'est contre-intuitif

Un préfixe `NEXT_PUBLIC_` signifie que la variable est **inscrite en dur dans le JavaScript envoyé au navigateur**. La définir dans un `ConfigMap` ou un `environment:` au démarrage du conteneur **n'a aucun effet** sur le code déjà compilé : il faut **reconstruire l'image**. C'est pourquoi le `ConfigMap` du frontend a été supprimé : il donnait l'illusion d'un réglage.

Exécution locale avec docker-compose

```yaml
services:
  api:
    build: {context: ., dockerfile: backend/Dockerfile}
    ports: ["8000:8000"]
    environment:
      - MODEL_REGISTRY_DIR=/app/model_registry
      - PROCESSORS_DIR=/app/artifacts/preprocessing
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      start_period: 40s
  frontend:
    build: {context: ., dockerfile: frontend/Dockerfile,
            args: {NEXT_PUBLIC_API_URL: "http://localhost:8000"}}   # figee au BUILD
    ports: ["3000:3000"]
    depends_on:
      api: {condition: service_healthy}     # attend que l'API soit SAINE
networks:
  fraud-net: {driver: bridge}
```

LE POINT REMARQUABLE

`depends_on: condition: service_healthy` : le frontend ne démarre pas avant que le *healthcheck* de l'API soit vert. C'est l'équivalent local de la **readinessProbe** Kubernetes.

Trois URL en développement : `localhost:3000` (interface), `localhost:8000/docs` (Swagger), `localhost:8000/metrics` (Prometheus).

Bonnes pratiques pour une API de modèle

À faire

- Charger le modèle au démarrage, jamais par requête.
- Valider les entrées avec des contraintes de domaine.
- Exposer `/health` **et** `/metrics`.
- Renvoyer la probabilité, pas seulement la classe.
- Versionner l'API (`/v1/predict`).
- Journaliser les erreurs avec un identifiant de corrélation.
- Prévoir un *timeout* et une limite de taille de requête.

À éviter

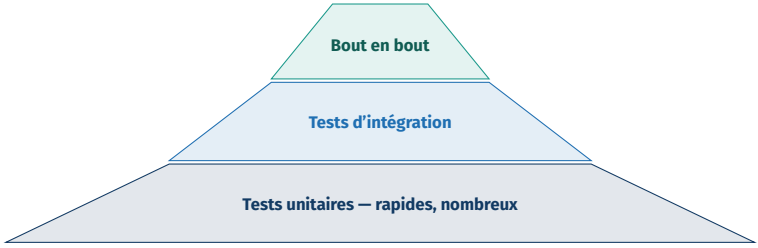
- Réécrire le preprocessing dans l'API.
- Laisser une exception remonter en `500` sans message utile.
- Coder un chemin absolu en dur comme valeur par défaut (corrigé : chemin relatif à la racine du dépôt).
- Exposer les endpoints d'administration sans protection (corrigé : `X-API-Token`).
- Renvoyer `200` quand le service n'est pas prêt (corrigé : `503`).
- Oublier la limite de taille sur `/predictCSV`.

En production, ajouter : authentification, limitation de débit (*rate limiting*), et journalisation des prédictions pour l'audit.

9

Testing : Pytest et Deepchecks

Que teste-t-on dans un système ML ?



les trois étages propres au ML

Tests de DONNÉES
schéma, valeurs, distributions

Tests de MODÈLE
seuils, non-régression, robustesse

Tests d'INFRA
sondes, latence, montée en charge

La pyramide classique reste valable, mais le ML ajoute **trois axes de test que le génie logiciel ne connaît pas** : les données peuvent être valides syntaxiquement et fausses statistiquement ; le modèle peut être « correct » et médiocre ; le système peut répondre et être trop lent.

Les quatre familles de tests, en pratique

Famille	Question posée	Outil	Quand
Code	La fonction Haversine renvoie-t-elle la bonne distance ?	Pytest	À chaque commit
Données	Le nouveau lot a-t-il le bon schéma, pas de valeurs aberrantes ?	Deepchecks	Avant l'entraînement
Modèle	Le ROC-AUC dépasse-t-il le seuil ? Est-il meilleur que le champion ?	Pytest + MLflow	Après l'entraînement
Système	L'API répond-elle correctement et assez vite ?	Pytest + TestClient	Avant le déploiement

LE PRINCIPE DU *QUALITY GATE*

Chaque famille est un **point de blocage** dans le pipeline : si le test échoue, la chaîne s'arrête et le modèle ne part pas en production.

> Un test qui n'a pas le pouvoir de **bloquer** n'est pas un test, c'est un rapport. La différence tient au code de sortie de la commande.

Pytest — les fondamentaux

```python
# fichier test_preprocessing.py  (prefixe "test_" -> decouverte automatique)
import pytest
from preprocessing import PreprocessingFraud

def test_haversine_distance_connue():
    """Paris -> Lyon : environ 392 km."""
    p = PreprocessingFraud()
    d = p._haversine_distance(48.8566, 2.3522, 45.7640, 4.8357)
    assert 390 < d < 395              # un assert, pas un print

def test_haversine_distance_nulle():
    p = PreprocessingFraud()
    assert p._haversine_distance(48.0, 2.0, 48.0, 2.0) == pytest.approx(0.0)

@pytest.mark.parametrize("heure,attendu", [(2,'nuit'), (9,'matin'), (21,'soiree')])
def test_periode_journee(heure, attendu):
    assert PreprocessingFraud()._get_period(heure) == attendu
```

`pytest -v` — détaillé `pytest -k haversine` — filtre
`pytest –cov=backend` — couverture

`@parametrize` évite quatre tests copiés-collés : **un cas de test = une ligne de données**.

Fixtures et *mocks* : indispensables en ML

```python
# --- Une fabrique : un faux modele qui renvoie la probabilite voulue ---
def fake_model(p):
    model = MagicMock()
    model.predict_proba.return_value = np.array([[1 - p, p]])
    return model

# --- Un mock de module (test_jenkins_scripts.py) : mlflow n'est pas joignable ---
for module in ("mlflow", "mlflow.sklearn", "mlflow.tracking"):
    sys.modules.setdefault(module, MagicMock())
from register_best_model import decide_promotion    # l'import ne plante plus

# --- Un patch cible : on remplace les globales du module api ---
@patch('api.preprocessor')
@patch('api.model', new_callable=lambda: fake_model(0.7))
def test_predict_applies_decision_threshold(mock_mod, mock_prep):
    mock_prep.preprocess_inference.return_value = pd.DataFrame([1])
    with patch('api.decision_threshold', 0.8):
        assert client.post("/predict", json=sample_data).json()["predictions"] == [0]
    with patch('api.decision_threshold', 0.6):
        assert client.post("/predict", json=sample_data).json()["predictions"] == [1]
```

Sans *mock*, ce test exigerait le vrai modèle, les artefacts DVC et un serveur MLflow joignable. Avec, la suite (23 tests) s'exécute en **quelques secondes** — donc à chaque commit.

Tester l'API sans serveur : `TestClient`

```python
from fastapi.testclient import TestClient
client = TestClient(app)                # aucun uvicorn lance, tout en memoire

def test_health_is_503_without_model():   # sans 'with' : le startup ne tourne pas
    r = client.get("/health")
    assert r.status_code == 503
    assert r.json()["status"] == "degraded"

def test_predict_csv_endpoint(...):
    files = {'file': ('test.csv', "amt,lat,long\n100,40,-74", 'text/csv')}
    r = client.post("/predictCSV", files=files)
    assert r.status_code == 200
    assert "text/csv" in r.headers["content-type"]
    assert "attachment" in r.headers["content-disposition"]
```

`TestClient` exécute l'application ASGI **en processus** : on teste le routage, la validation Pydantic, les codes de statut et les en-têtes HTTP, sans réseau ni conteneur — le test d'intégration le moins coûteux qui existe.

Le test de scénario métier : 11 fraudes réelles

```python
MIN_DETECTED = int(os.getenv("MIN_DETECTED", "9"))

def run_scenario(min_detected=MIN_DETECTED):
    """11 fraudes issues du TEST set, jamais vues par le modele."""
    model     = pickle.load(open(MODEL_PATH, "rb"))           # production.pkl
    threshold = resolve_threshold(json.loads(METADATA_PATH.read_text()))  # regle de l'API
    X     = InferencePreprocessor().preprocess_inference(pd.DataFrame(FRAUDS))
    preds = (model.predict_proba(X)[:, 1] >= threshold).astype(int)
    detected = int(preds.sum())
    result = {"detected": detected, "total": len(FRAUDS), "passed": detected >= min_detected}
    RESULT_PATH.write_text(json.dumps(result))     # lu par push_metrics.py -> Pushgateway
    return result

if __name__ == "__main__":                         # stage Jenkins 'Scenario Test'
    sys.exit(0 if run_scenario()["passed"] else 1)
```

Faiblesse corrigée dans cette version

Avant : `assert len(results) == 11` vérifiait seulement que **le pipeline s'exécute** — le test passait même avec **zéro** fraude détectée. **Après** : `detected >= MIN_DETECTED` (9 sur 11), un **seuil de rappel minimal** qui fait échouer le build et bloque le déploiement.

Tests spécifiques au modèle

Type de test	Principe	Exemple pour la fraude
Seuil minimal	Refuser un modèle sous un plancher	`assert roc_auc > 0.95`
Non-régression	Le candidat doit battre le champion	`assert new_auc >= old_auc`
Invariance	Une modification neutre ne doit pas changer la prédiction	Changer `job` ne doit pas basculer la décision
Directionnalité	Une modification orientée doit pousser dans le bon sens	Montant $\times$ 10 la nuit $\longrightarrow$ probabilité $\nearrow$
Cas limites	Robustesse aux valeurs extrêmes	`amt = 0`, distance $=$ 0, âge $=$ 100 ans
Équité	Pas de disparité injustifiée entre groupes	Taux de faux positifs par genre
Latence	Respect de la contrainte temps réel	`assert duration < 0.2`

Les tests d'**invariance** et de **directionnalité** (approche *behavioral testing*) sont les plus utiles en pratique : ils vérifient que le modèle a appris **le bon raisonnement**, et pas seulement la bonne réponse moyenne.

Deepchecks — valider les données, pas le code



doublons, valeurs manquantes, types

dérive, fuite, nouvelles catégories

sur-apprentissage, segments faibles

rapport HTML complet

LE PRINCIPE

Deepchecks fournit des **dizaines de contrôles préécrits**, regroupés en suites, avec des seuils par défaut et un rapport HTML interactif.

Chaque *check* renvoie un statut ✓ / ⚠ / ✗ **et** une valeur, ce qui permet de décider par programme si le pipeline continue.

Deepchecks — déclarer le dataset

```python
from deepchecks.tabular import Dataset
from deepchecks.tabular.suites import full_suite

df = pd.read_csv("../data/raws/fraud_600k.csv").drop(['Unnamed: 0'], axis=1)

# Deepchecks a besoin de savoir CE QU'EST chaque colonne
cat_features = df.select_dtypes(include=['object']).columns.tolist()

ds = Dataset(df=df,
             label='is_fraud',          # la cible
             cat_features=cat_features) # les colonnes categorielles

result = full_suite().run(ds)           # execute toute la suite
result.save_as_html('artifacts/deepchecks_test/result_integrity_test.html')
```

POURQUOI L'OBJET DATASET

Un `DataFrame` ne sait pas quelle colonne est la cible, ni lesquelles sont catégorielles. `Dataset` **ajoute cette sémantique**, ce qui permet à Deepchecks de choisir les bons tests statistiques.

Sur un jeu de 600 000 lignes, la `full_suite` est lente. En CI, on privilégie une **suite ciblée** sur un **échantillon** : `data_integrity()` sur 50 000 lignes.

Ce que détecte la suite `data_integrity`

Check	Ce qu'il cherche	Pourquoi c'est grave
Data Duplicates	Lignes strictement identiques	Biaise les métriques, gonfle le train
Mixed Data Types	Une colonne mêlant nombres et chaînes	Casse l'encodage silencieusement
String Mismatch	`"NY"`, `"ny"`, `"N.Y."`	Trois catégories pour une seule réalité
Conflicting Labels	Mêmes features, étiquettes opposées	Plafonne la performance atteignable
Outlier Sample	Points très atypiques	Peuvent dominer l'apprentissage
Feature-Label Correlation	Corrélation quasi parfaite	**Signal de fuite de données**
Single Value in Column	Colonne constante	Inutile, occupe de la place
Special Characters	Caractères de contrôle, encodage cassé	Erreurs à l'inférence
Class Imbalance	Déséquilibre de la cible	Impose SMOTE ou pondération

> Le check **Feature-Label Correlation** est le plus précieux : c'est le détecteur automatique de fuite de données. Une corrélation de 0,99 entre une feature et la cible signifie presque toujours une **erreur de construction du dataset**.

Comparer deux jeux : `train_test_validation`

```
df_train, df_test = train_test_split(dfcopy, test_size=0.2, random_state=42)

ds_train = Dataset(df=df_train, label='is_fraud', cat_features=cat_features)
ds_test  = Dataset(df=df_test,  label='is_fraud', cat_features=cat_features)

result = full_suite().run(train_dataset=ds_train, test_dataset=ds_test)
```

Check	Ce qu'il révèle
Train Test Feature Drift	Une feature suit une distribution différente entre train et test
Train Test Label Drift	Le taux de fraude diffère entre les deux jeux
New Category	Une modalité présente en test et absente du train → l'encodeur échouera
Train Test Samples Mix	Des lignes du test présentes dans le train → **fuite directe**
Index/Datetime Leakage	Le split n'est pas cohérent temporellement
Whole Dataset Drift	Un modèle arrive-t-il à distinguer train et test ? Si oui, ils diffèrent

Le même outillage servira, en production, à comparer **données de référence** et **données entrantes** — c'est la transition naturelle vers Evidently AI (section 13).

La suite `model_evaluation`

Check	Ce qu'il apporte au-delà d'un score global
Simple Model Comparison	Le modèle bat-il une baseline triviale (classe majoritaire) ?
Train Test Performance	Écart train/test : mesure directe du sur-apprentissage
Weak Segments Performance	**Identifie les sous-populations mal traitées**
Confusion Matrix Report	Répartition détaillée VP/FP/VN/FN
ROC Report	Courbe et seuils optimaux par classe
Calibration Score	Les probabilités prédites sont-elles fiables ?
Prediction Drift	La distribution des prédictions a-t-elle changé ?
Boosting Overfit	Détecte le sur-apprentissage lié au nombre d'itérations

Weak Segments Performance est le plus utile en fraude : un modèle peut afficher 0,999 de ROC-AUC global et **échouer complètement** sur les transactions à petit montant. Un score agrégé masque ce genre de défaillance ; ce check l'expose.

Intégrer Deepchecks dans la CI

```python
# variante Deepchecks d'un controle appele par Jenkins AVANT l'entrainement
from deepchecks.tabular.suites import data_integrity

result = data_integrity().run(ds)
result.save_as_html("artifacts/deepchecks/integrity.html")   # trace archivable

failed = [c for c in result.get_not_passed_checks()]
if failed:
    print(f"{len(failed)} controles en echec :")
    for c in failed:
        print("  -", c.header)
    sys.exit(1)          # <-- code de sortie non nul : Jenkins arrete le pipeline
print("Validation des donnees : OK")
```

LA LIGNE QUI CHANGE TOUT

`sys.exit(1)`. Sans elle, le rapport HTML est produit et **personne ne le lit**. Avec elle, la validation des données devient un véritable *quality gate*.

Dans notre projet

Notre stage 3 lance `validate_data.py` : contrôles **pandas** légers, avec ce même `sys.exit(1)`. Deepchecks reste en notebook : l'y brancher est l'étape suivante.

Stratégie de test : que lancer, et quand

Moment	Tests exécutés	Durée cible	Si échec
Avant commit (local)	Tests unitaires rapides	< 10 s	Le commit est corrigé
À chaque *push*	Unitaires + lint + `TestClient`	< 2 min	La CI échoue, pas de fusion
Avant l'entraînement	Intégrité et dérive des données	< 5 min	Pas d'entraînement lancé
Après l'entraînement	Seuils, non-régression, comportement	< 5 min	Pas de promotion du modèle
Avant le déploiement	Scénarios métier, latence	< 3 min	Pas de déploiement
Après le déploiement	*Smoke test* sur `/health` et `/predict`	< 30 s	Rollback automatique

Règle de conception : **plus un test est lent, plus tard il intervient**. Un test unitaire qui prend deux minutes ne sera jamais lancé par les développeurs — et cessera donc d'être utile.

10

Conteneurisation avec Docker

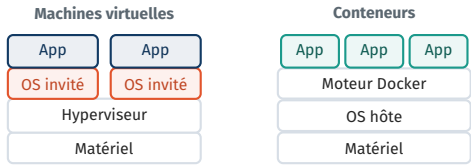
Le problème que Docker résout

LES SOURCES D'ÉCART ENTRE ENVIRONNEMENTS

- Version de Python (3.10 en local, 3.12 sur le serveur).
- Version de `scikit-learn` : un `.pkl` sérialisé avec la 1.6 ne se recharge pas toujours avec la 1.3.
- Bibliothèques système : `libgomp1` est requis par LightGBM.
- Variables d'environnement, chemins, encodage, fuseau horaire.

CE QU'APPORTE UNE IMAGE

Le code, l'interpréteur, les dépendances Python *et* les bibliothèques système sont **figés ensemble** dans un artefact unique et immuable.



Un conteneur partage le noyau de l'hôte : démarrage en **millisecondes**, empreinte de quelques dizaines de Mo, contre des Go et des dizaines de secondes pour une VM.

Le vocabulaire Docker

Terme	Définition	Dans notre projet
Dockerfile	La recette de construction	`backend/Dockerfile`
Image	Le résultat, en lecture seule, en couches	`<compte>/fraud-backend:latest`
Couche	Le delta produit par une instruction	Mutualisée entre images
Conteneur	Une instance en cours d'exécution	Un pod Kubernetes en contient un
Registry	Le dépôt d'images	Docker Hub
Tag	L'étiquette de version	`:latest`, `:1.0.0`, `:$GIT_SHA`
Volume	Un stockage persistant hors image	Non utilisé (modèle embarqué)

Le tag `:latest` est un piège

`:latest` n'est pas une version, c'est un **alias mouvant**. Deux pods démarrés à cinq minutes d'intervalle peuvent exécuter deux images différentes, et il devient impossible de savoir quel modèle a produit une prédiction donnée.
Bonne pratique : tagger avec le SHA du commit et laisser `:latest` comme simple alias de commodité.

Les instructions d'un Dockerfile

```dockerfile
FROM python:3.12-slim          # image de base : le point de depart
WORKDIR /app                   # repertoire de travail par defaut
COPY requirements.txt .        # copie hote -> image
RUN pip install -r requirements.txt   # execute AU BUILD, cree une couche
ENV PYTHONUNBUFFERED=1         # variable d'environnement persistante
ARG API_URL=http://localhost   # variable disponible AU BUILD seulement
EXPOSE 8000                    # documentation du port (n'ouvre rien)
USER appuser                   # utilisateur d'execution
HEALTHCHECK CMD curl -f http://localhost:8000/health || exit 1
CMD ["uvicorn", "app:app", "--host", "0.0.0.0"]   # commande AU RUN
```

DEUX DISTINCTIONS À RETENIR

- `RUN` s'exécute au **build** ; `CMD` au **démarrage**.
- `ARG` n'existe qu'au build ; `ENV` persiste dans le conteneur.

`–host 0.0.0.0` est obligatoire : avec `127.0.0.1`, le serveur n'écoute que l'intérieur du conteneur et **aucune requête externe n'aboutit**. Erreur très fréquente.

Le cache de couches : l'ordre des instructions compte

✗ MAUVAIS ORDRE

```
COPY . .
RUN pip install -r requirements.txt
```

Toute modification d'une ligne de code invalide la couche `COPY`, donc **réinstalle toutes les dépendances**. Build de 6 minutes à chaque commit.

✓ BON ORDRE

```
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

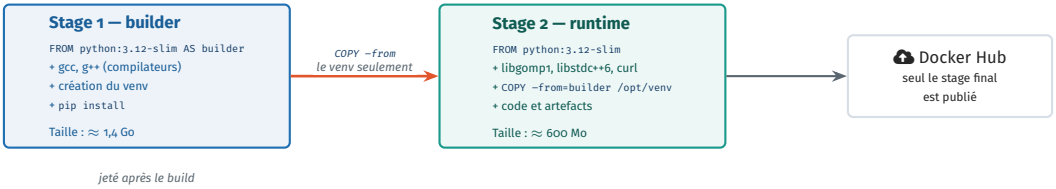
Les dépendances ne sont réinstallées que si `requirements.txt` change. Build de **15 secondes** pour une modification de code.



une couche invalidée invalide toutes les suivantes

Règle : **du plus stable au plus volatil**. Le code applicatif se copie **en dernier**.

Le build multi-stage



- Les **compilateurs** servent à installer LightGBM et XGBoost, mais sont inutiles pour exécuter le service.
- L'image finale contient **l'environnement virtuel construit**, pas les outils qui l'ont construit.
- Bénéfices : image plus légère, **surface d'attaque réduite**, téléchargement plus rapide sur chaque nœud du cluster.

Notre Dockerfile backend — stage 1

```dockerfile
# ============ Stage 1 : builder ============
FROM python:3.12-slim AS builder
WORKDIR /build

# Compilateurs necessaires a certains paquets scientifiques
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc g++ \
    && rm -rf /var/lib/apt/lists/*          # <- on nettoie DANS la meme couche

COPY backend/requirements.txt .

RUN python -m venv /opt/venv                # environnement isole et copiable
ENV PATH="/opt/venv/bin:$PATH"

RUN pip install --default-timeout=1000 --no-cache-dir --upgrade pip && \
    pip install --default-timeout=1000 --no-cache-dir -r requirements.txt
```

- `rm -rf /var/lib/apt/lists/*` dans le *même* `RUN` : sinon les fichiers restent dans la couche précédente et **la taille n'est pas réduite**.
- `–no-cache-dir` : évite d'embarquer le cache pip (≈ 300 Mo pour cette liste de dépendances).
- `venv` dans `/opt/venv` : un répertoire unique, facile à copier vers le stage suivant.

Notre Dockerfile backend — stage 2

```dockerfile
FROM python:3.12-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
    libgomp1 libstdc++6 curl && rm -rf /var/lib/apt/lists/*   # runtime LightGBM + curl

RUN groupadd -r appuser && useradd -r -g appuser -u 1001 appuser   # non-root
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv                            # le venv construit
COPY --chown=appuser:appuser backend/src/ /app/backend/src/

# Uniquement les processeurs necessaires a l'inference (scaler, encoders, features)
COPY --chown=appuser:appuser artifacts/preprocessing/*.pkl /app/artifacts/preprocessing/
# Uniquement le meilleur modele -- pas les 10 modeles entraines
COPY --chown=appuser:appuser artifacts/mlflow/Best_Fraud_LightGBM/1.0.0/ /app/model_registry/

ENV PATH="/opt/venv/bin:$PATH" PYTHONPATH="/app/backend/src:/app" \
    MODEL_REGISTRY_DIR=/app/model_registry PROCESSORS_DIR=/app/artifacts/preprocessing
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1
USER appuser
CMD ["uvicorn", "backend.src.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Copier **sélectivement** les artefacts est essentiel : le dossier complet pèse 2,3 Go, l'image n'embarque que ce qui sert à **prédire**.

Le Dockerfile frontend : trois étages

```dockerfile
# --- deps : installer les dependances Node ---
FROM node:18-alpine AS deps
COPY frontend/package.json frontend/package-lock.json* ./
RUN npm ci                          # ci (et non install) : respecte le lockfile

# --- builder : compiler l'application Next.js ---
FROM node:18-alpine AS builder
COPY --from=deps /app/node_modules ./node_modules
COPY frontend/ .
ARG NEXT_PUBLIC_API_URL=http://localhost:8000
ENV NEXT_PUBLIC_API_URL=$NEXT_PUBLIC_API_URL
RUN npm run build                   # produit .next/standalone

# --- runner : image finale minimale ---
FROM node:18-alpine AS runner
RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nextjs
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./.next/static
ENV NODE_ENV=production PORT=3000 HOSTNAME="0.0.0.0"
USER nextjs
CMD ["node", "server.js"]
```

`output: 'standalone'` produit un serveur Node autonome : l'image finale ne contient **ni `node_modules` complet ni sources** — de $\approx$ 1,2 Go à $\approx$ 150 Mo.

Le `.dockerignore` : critique dans ce projet

```
# Gros artefacts inutiles a l'image
artifacts/modeling/
artifacts/eda/
artifacts/deepchecks_test/
artifacts/mlflow/models/
artifacts/jenkins/
artifacts/preprocessing/preprocessed_data.pkl
data/                          # tout le dataset

# Environnements et caches
**/node_modules
**/__pycache__
**/.venv
frontend/.next

# Secrets et metadonnees
.dvc/
.git
*.log
**/*.ipynb
test/
```

POURQUOI C'EST DÉCISIF ICI

Le contexte de build est la **racine du dépôt**, qui contient **2,3 Go d'artefacts** et le dataset. Sans `.dockerignore`, le démon Docker reçoit tout ce volume **avant même** de lire la première instruction.

`.git` et `.dvc/` sont exclus — et c'est aussi une mesure de **sécurité** : `.dvc/config` peut contenir des identifiants, et `.git` contient tout l'historique.

Vérification : `docker build` affiche « *Sending build context* » — au-delà de quelques Mo, le `.dockerignore` est à revoir.

Les commandes du quotidien

```
# --- Construire ---
TAG="${BUILD_NUMBER}-$(git rev-parse --short=7 HEAD)"     # ex. 42-a1b2c3d
docker build -t <compte>/fraud-backend:$TAG -f backend/Dockerfile .
docker build --build-arg NEXT_PUBLIC_API_URL=http://api.fraud.local \
             -t <compte>/fraud-frontend:$TAG -f frontend/Dockerfile .

# --- Executer et inspecter ---
docker run -p 8000:8000 --env-file .env <compte>/fraud-backend:$TAG
docker logs -f fraud_detection_api          # suivre les journaux
docker exec -it fraud_detection_api /bin/bash   # ouvrir un shell dedans
docker inspect fraud_detection_api | grep -i health

# --- Publier ---
echo "$PWD_TOKEN" | docker login -u "$USER" --password-stdin
docker push <compte>/fraud-backend:$TAG     # tag immuable : la version deployee

# --- Diagnostiquer et nettoyer ---
docker images                  # tailles des images
docker history <image>         # poids de chaque couche
docker system prune -af        # liberer l'espace disque
```

`docker history` est l'outil de diagnostic clé : il montre **quelle instruction** fait grossir l'image.

Bonnes pratiques et anti-patterns Docker

À faire

- Partir d'une image `-slim` ou `-alpine`.
- Épingler la version de base (`python:3.12-slim`, jamais `python`).
- Exécuter en **utilisateur non-root**.
- Multi-stage systématique.
- Un `.dockerignore` strict.
- Un `HEALTHCHECK`.
- Taguer avec le SHA du commit.
- Scanner les images (`docker scout`, Trivy).

À ne pas faire

- Mettre un secret dans le Dockerfile (il reste dans les couches, même supprimé ensuite).
- Utiliser `ARG` pour un mot de passe : il est visible dans `docker history`.
- Lancer plusieurs processus dans un conteneur.
- Stocker des données dans le conteneur (il est éphémère).
- `COPY . .` en première instruction.
- Se reposer sur `:latest` en production.
- Écouter sur `127.0.0.1`.

Un conteneur est **immuable et jetable** : toute donnée qui doit survivre à son arrêt vit ailleurs (volume, base, stockage objet).

11

CI/CD avec Jenkins

Le rôle de l'orchestrateur

CE QU'UN ORCHESTRATEUR CI/CD APPORTE

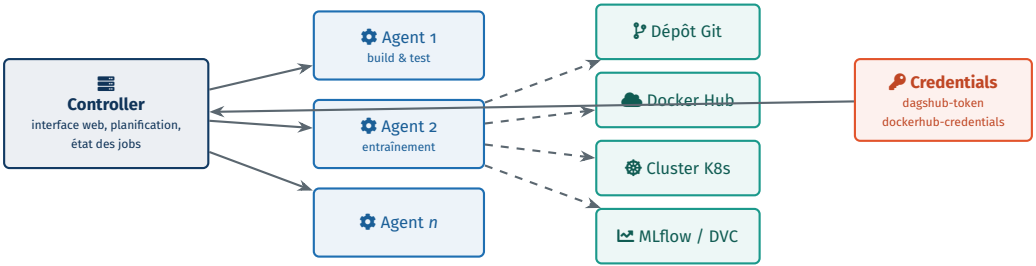
- Un **déclencheur** : commit, horaire, événement, appel d'un autre pipeline.
- Un **enchaînement** d'étapes avec conditions et parallélisme.
- Un **environnement d'exécution** reproductible (l'agent).
- La **gestion des secrets**.
- Une **trace** : journaux, artefacts archivés, historique des exécutions.
- Des **notifications** en cas d'échec.

Pourquoi Jenkins dans ce projet

- **Auto-hébergé** : il tourne dans le même réseau que le cluster Minikube, ce qu'un service SaaS ne peut pas faire simplement.
- **Contrôle total de l'agent** : on y installe Python 3.11, DVC, `kubectl` et le client Docker.
- **Longues exécutions** acceptées : un entraînement de 20 minutes ne pose pas de problème de quota.

Contrepartie : c'est **une infrastructure à maintenir** (mises à jour, sauvegardes, sécurité). GitHub Actions supprime ce coût mais complique l'accès à un cluster local.

Architecture de Jenkins



- Le **controller** ne doit **jamais** exécuter les builds : il planifie et conserve l'état.
- Les **agents** exécutent le travail et sont jetables.
- **Notre configuration** : `agent any`, un seul nœud — acceptable en projet pédagogique, à séparer en production.

Construire l'agent : le Dockerfile Jenkins

```
FROM jenkins/jenkins:lts
USER root

# 1. Python 3.11 compile depuis les sources (l'image de base n'en a pas)
RUN wget https://www.python.org/ftp/python/3.11.9/Python-3.11.9.tgz && \
    tar -xf Python-3.11.9.tgz && cd Python-3.11.9 && \
    ./configure --enable-optimizations && make -j$(nproc) && make altinstall

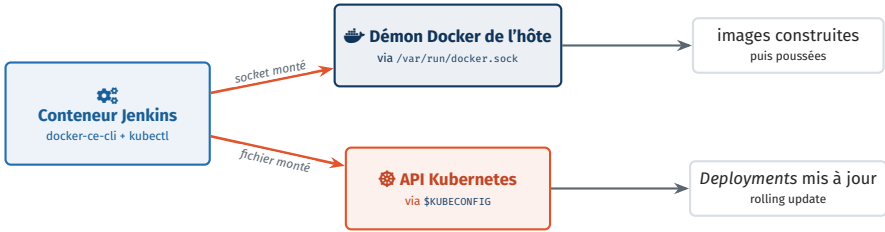
# 2. Le CLIENT Docker (pas le demon) pour piloter le demon de l'hote
RUN apt-get install -y docker-ce-cli docker-compose-plugin

# 3. kubectl pour deployer sur le cluster
RUN curl -LO "https://dl.k8s.io/release/v1.32.2/bin/linux/amd64/kubectl" && \
    chmod +x kubectl && mv kubectl /usr/local/bin/kubectl

# 4. Environnement virtuel PRE-INSTALLE : evite 6 min de pip a chaque build
COPY requirements.txt /tmp/requirements.txt
RUN python3.11 -m venv /opt/venv-fraud && \
    /opt/venv-fraud/bin/pip install -r /tmp/requirements.txt \
        "evidently>=0.7,<0.8"       # dvc, mlflow, prometheus-client : deja requis
USER jenkins
```

Le point nº 4 est la **principale optimisation** : les dépendances ML sont dans l'image de l'agent, chaque *stage* se contente d'un `. /opt/venv-fraud/bin/activate`.

Comment Jenkins pilote Docker et Kubernetes



DOCKER OUTSIDE OF DOCKER

Jenkins n'exécute pas son propre démon Docker : il utilise le **socket du démon de l'hôte**. Les images construites sont donc directement visibles par la machine hôte.

Monter `/var/run/docker.sock` donne à Jenkins un **contrôle total de l'hôte** : quiconque peut modifier un `Jenkinsfile` peut exécuter du code privilégié. À isoler strictement en production.

Anatomie d'un pipeline déclaratif

```
pipeline {
    agent any                          // ou : agent { docker { image '...' } }

    environment {                      // variables visibles par tous les stages
        DAGSHUB_TOKEN       = credentials('dagshub-token')
        MLFLOW_TRACKING_URI = 'https://dagshub.com/<utilisateur>/<depot>.mlflow'
        PUSHGATEWAY_URL     = 'http://192.168.49.2:30091'    // lu par push_metrics.py
    }

    stages {
        stage('Checkout')        { steps { retry(3) { checkout scm } } }
        stage('Data Drift')      { steps { sh 'python3 jenkins/drift_detection.py' } }
        stage('Training')        { when { expression { env.DRIFT == 'true' } }
                                   steps { sh 'python3 jenkins/train_model.py' } }
    }

    post {                             // s'execute TOUJOURS, quel que soit le resultat
        always  { archiveArtifacts artifacts: 'artifacts/jenkins/*.html' }
        success { echo 'Pipeline OK' }
        failure { sh 'python3 jenkins/push_metrics.py 0' }
    }
}
```

`retry(3)` sur le `checkout` : les échecs réseau transitoires sont la première cause de builds rouges non significatifs.

La gestion des secrets

```
environment {
    DAGSHUB_TOKEN         = credentials('dagshub-token')      // secret simple
    DOCKERHUB_CREDENTIALS = credentials('dockerhub-credentials')  // secret compose
    //   -> cree DOCKERHUB_CREDENTIALS_USR  et  DOCKERHUB_CREDENTIALS_PSW
}
steps {
    sh '''
        echo "${DOCKERHUB_CREDENTIALS_PSW}" | \
        docker login -u "${DOCKERHUB_CREDENTIALS_USR}" --password-stdin
    '''
}
```

DEUX PROTECTIONS IMPORTANTES

- Jenkins **masque** automatiquement ces valeurs dans les journaux.
- `–password-stdin` évite que le mot de passe apparaisse dans la liste des processus.

Le masquage est **contourné** par `echo $TOKEN` ou par une commande qui affiche son environnement. Ne jamais faire confiance au seul masquage : ne pas afficher les secrets, tout court.

Pipeline nº 1 — le pipeline Machine Learning



- Les stages 5, 6, 7 et 9 sont **conditionnels** : sans dérive, on ne ré-entraîne pas. Le déploiement (9) exige **en plus** que le challenger ait battu le champion.
- Les stages 3 et 7 sont des **quality gates** : données invalides ou moins de 9 fraudes détectées sur 11 → build rouge, rien n'est déployé.
- Le stage 8 s'exécute **toujours** : le tableau de bord doit afficher le score de dérive même quand il est bas.

Stages 1 à 3 — code, données et validation

```
stage('Checkout') { steps {
    retry(3) { checkout scm }
    sh 'rm -f artifacts/jenkins/*.txt artifacts/jenkins/*.json artifacts/jenkins/*.html' } }
stage('Setup Environment') { steps { sh '''
    . /opt/venv-fraud/bin/activate
    dvc remote modify origin --local user "${DAGSHUB_USER}"
    dvc remote modify origin --local password "${DAGSHUB_TOKEN}"
    dvc pull data/raws/fraud_600k.csv artifacts/preprocessing/ --jobs 1
    dvc pull data/raws/new_data.csv --jobs 1 || echo "Pas de new_data.csv dans le remote" ''' } }
stage('Data Validation') {               // quality gate : exit 1 -> le pipeline s'arrete
    steps { sh '. /opt/venv-fraud/bin/activate && python3 jenkins/validate_data.py' } }
```

TROIS DÉTAILS QUI COMPTENT

- `rm -f artifacts/jenkins/*` : sans cela, une dérive ou une promotion de l'exécution précédente pourrait être relue.
- `validate_data.py` : schéma, valeurs nulles, bornes, catégories connues — **avant** tout ré-entraînement.
- `dvc pull` **ciblé** : inutile de rapatrier 2,3 Go quand trois chemins suffisent.

L'espace de travail Jenkins n'est **pas garanti propre** entre deux builds. Tout pipeline fiable commence par nettoyer ce dont il dépend.

Stage 4 — faire circuler une décision entre stages

```
stage('Data Drift Detection') {
    steps { script {
        sh '. /opt/venv-fraud/bin/activate && python3 jenkins/drift_detection.py'
        // le script Python a ecrit artifacts/jenkins/drift_result.txt ("true" / "false")
        env.DRIFT_DETECTED = readFile('artifacts/jenkins/drift_result.txt').trim()
        echo "DRIFT_DETECTED = '${env.DRIFT_DETECTED}'"   // <- devient global au pipeline
    } }
}
```

Pourquoi passer par un fichier

Chaque `sh` est un **shell distinct** : une variable exportée dans un `sh` disparaît au suivant. Le motif est donc : **le script Python écrit un fichier** → **Groovy le lit** (`readFile`) → **la valeur va dans `env`**. Le `.trim()` supprime le retour à la ligne, sans quoi la comparaison `== 'true'` échouerait sur `"true\n"`. Le stage 6 réutilise exactement ce motif avec `promotion.txt`.

Stages 5 à 7 — exécution conditionnelle et quality gate

```
stage('Continuous Training') {      // MLFLOW_TRACKING_* : bloc environment (credentials)
    when { expression { env.DRIFT_DETECTED == 'true' } }
    steps { sh '. /opt/venv-fraud/bin/activate && python3 jenkins/train_model.py' }
}

stage('Champion / Challenger') {
    when { expression { env.DRIFT_DETECTED == 'true' } }
    steps { script {
        sh '. /opt/venv-fraud/bin/activate && python3 jenkins/register_best_model.py'
        env.MODEL_PROMOTED = readFile('artifacts/jenkins/promotion.txt').trim()
    } }
}

stage('Scenario Test') {            // bloquant : moins de MIN_DETECTED (9) fraudes sur 11 -> exit 1
    when { expression { env.DRIFT_DETECTED == 'true' } }
    steps { sh '. /opt/venv-fraud/bin/activate && python3 test/test_fraud_scenario.py' }
}
```

Les stages sautés apparaissent en gris dans l'interface Jenkins : on voit **immédiatement** qu'il n'y a pas eu de dérive, sans lire les journaux. Si le challenger ne bat pas le champion, `MODEL_PROMOTED` vaut `false` et rien n'est déployé.

Stages 8 et 9, bloc post

```
stage('Publish Drift to Backend') {           // sans condition : toujours execute
    // POST /update-drift-metrics + /upload-drift-report (en-tete X-API-Token si configure)
    steps { sh '. /opt/venv-fraud/bin/activate && python3 jenkins/publish_drift.py' }
}
stage('Trigger Deploy Pipeline') {            // seulement si un NOUVEAU champion a ete promu
    when { expression { env.DRIFT_DETECTED == 'true' && env.MODEL_PROMOTED == 'true' } }
    steps { build job: 'fraud-deploy-pipeline', wait: true }
}
post {
    always  { archiveArtifacts artifacts: 'artifacts/jenkins/*.html, artifacts/jenkins/*.json',
                               allowEmptyArchive: true }
    success { sh '. /opt/venv-fraud/bin/activate && python3 jenkins/push_metrics.py 1' }
    failure { sh '. /opt/venv-fraud/bin/activate && python3 jenkins/push_metrics.py 0' }
}
```

`push_metrics.py` pousse vers le *Pushgateway* le statut du build, le résultat du test de scénario et la décision de promotion : **la santé du pipeline devient une métrique observable**, avec son alerte `MLPipelineFailed`, au même titre que la latence de l'API.

Pipeline nº 2 — le pipeline de déploiement

```
env.IMAGE_TAG = "${env.BUILD_NUMBER}-${GIT_SHA}"    // ex. 42-a1b2c3d : immuable et tracable

stage('Fetch Champion Model') { steps {              // le registre MLflow fait foi, pas DVC
    sh '. /opt/venv-fraud/bin/activate && python3 jenkins/register_best_model.py --export-champion' } }
stage('Build & Push Images') {
    parallel {                                       // backend et frontend en simultane
        stage('Backend') { steps { sh '''
            docker build -t ${IMAGE_BACKEND}:${IMAGE_TAG} -t ${IMAGE_BACKEND}:latest -f backend/Dockerfile .
            docker push ${IMAGE_BACKEND}:${IMAGE_TAG} && docker push ${IMAGE_BACKEND}:latest ''' } }
        stage('Frontend') { steps { sh '... idem, avec --build-arg NEXT_PUBLIC_API_URL ...' } }
    }
}
stage('Deploy to Kubernetes') { steps { sh '''
    kubectl set image deployment/fraud-backend fraud-backend=${IMAGE_BACKEND}:${IMAGE_TAG} -n ${K8S_NAMESPACE}
    kubectl annotate deployment/fraud-backend kubernetes.io/change-cause="build ${BUILD_NUMBER}" --overwrite '''
      } }
stage('Rollout Status') { steps { sh '''
    kubectl rollout status deployment/fraud-backend -n ${K8S_NAMESPACE} --timeout=300s \
      || { kubectl rollout undo deployment/fraud-backend -n ${K8S_NAMESPACE}; exit 1; } ''' } }
```

Le tag change à chaque build, donc la spec du Deployment change et Kubernetes lance **réellement** un *rolling update* — avec `:latest`, `set image` ne modifiait rien. Si les pods ne sont pas prêts en 300 s, `rollout undo` **restaure la version précédente** et le build échoue.

Les déclencheurs et le chaînage de pipelines

Déclencheur	Syntaxe	Usage pertinent
Sondage du dépôt	`pollSCM('H/5 * * * *')`	Simple, mais latence et charge
Webhook	Configuré côté GitHub	**Recommandé** : réaction immédiate
Planification	`cron('H 2 * * *')`	Ré-entraînement nocturne
Pipeline amont	`build job: 'fraud-deploy-pipeline'`	**Notre chaînage ML → deploy**
Manuel	Bouton *Build Now*	Validation avant mise en production
Appel externe	Jeton d'API HTTP	Déclenchement par un événement métier

NOTRE CHAÎNAGE

`build job: 'fraud-deploy-pipeline', wait: true` — le pipeline ML attend la fin du déploiement, donc **un échec de déploiement rend le pipeline ML rouge**. La chaîne complète a un statut unique.

Le `H` des expressions cron signifie *hash* : Jenkins répartit les exécutions dans l'intervalle pour éviter que tous les jobs démarrent à la même seconde.

Bonnes pratiques CI/CD pour le ML

À faire

- Versionner le `Jenkinsfile` **avec** le code.
- Rendre chaque stage idempotent.
- Nettoyer l'espace de travail en début de pipeline.
- Archiver les rapports (dérive, tests, couverture).
- Séparer pipeline ML et pipeline de déploiement.
- Utiliser `parallel` pour les tâches indépendantes.
- Exposer le statut du build comme métrique.
- Fixer un *timeout* global au pipeline.

À éviter

- Configurer le job dans l'interface web (rien n'est versionné).
- Disperser les adresses IP dans les scripts (les nôtres sont désormais regroupées dans le bloc `environment`).
- Enchaîner sans *quality gate*.
- Laisser un pipeline s'exécuter des heures sans limite.
- Écrire des secrets dans les journaux.
- Déployer sans vérifier le *rollout*.
- Un pipeline unique et monolithique de 20 stages.

Test décisif : **un nouvel arrivant peut-il reconstruire toute la chaîne à partir du seul dépôt ?** Si la réponse est non, une partie de la configuration vit hors du versionnement.

12

Orchestration avec Kubernetes

Pourquoi un orchestrateur

CE QUE DOCKER RUN NE SAIT PAS FAIRE

- Redémarrer un conteneur mort.
- Le déplacer si la machine tombe.
- Passer de 1 à 10 instances selon la charge.
- Répartir le trafic entre ces instances.
- Remplacer une version **sans coupure**.
- Revenir en arrière en une commande.
- Vérifier que le service est *prêt* avant de l'exposer.

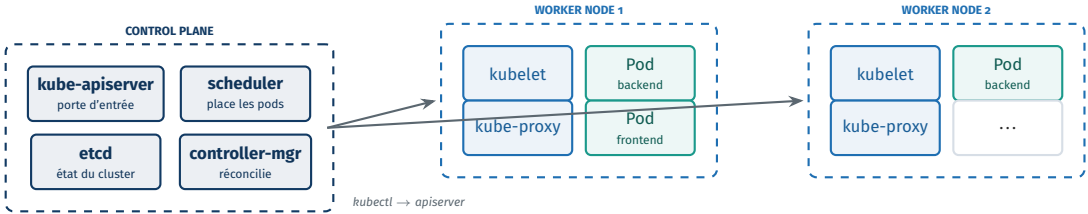
Le principe fondateur

Kubernetes est un système **déclaratif** : on décrit l'**état souhaité** (« je veux 3 répliques de cette image ») et un ensemble de contrôleurs travaillent en continu pour que l'**état réel** y corresponde.

Conséquence concrète : si l'on supprime un pod à la main, Kubernetes **le recrée**. Il n'y a pas de « démarrer » ni « arrêter », il y a **déclarer** et **réconcilier**.

Le prix : une courbe d'apprentissage réelle et un vocabulaire dense. C'est justifiifié dès qu'il faut de la disponibilité.

Architecture d'un cluster



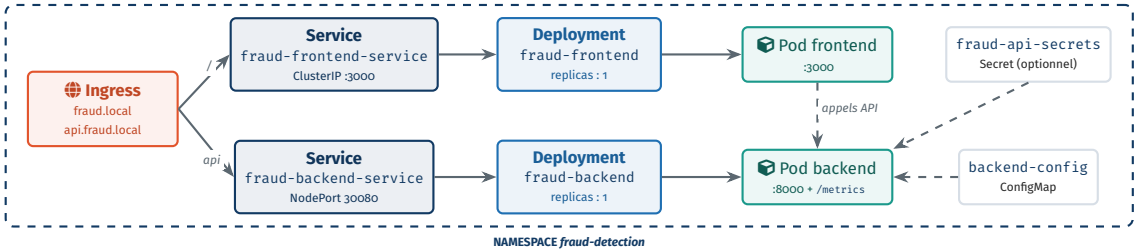
- Toute commande `kubectl` parle à l'**apiserver**, jamais aux nœuds.
- Le **kubelet** de chaque nœud démarre les conteneurs et remonte leur état.
- **Minikube** réunit control plane et worker sur une seule machine : suffisant pour développer, insuffisant pour la haute disponibilité.

Les objets essentiels

Objet	Rôle	Dans notre projet
Namespace	Cloisonnement logique et quotas	`fraud-detection`
Pod	La plus petite unité déployable (1..n conteneurs)	1 conteneur par pod
ReplicaSet	Maintient *n* pods identiques	Créé par le Deployment
Deployment	Gère les ReplicaSets, les mises à jour et le rollback	`fraud-backend`, `fraud-frontend`
Service	Adresse stable et répartition de charge	`fraud-backend-service` (NodePort)
Ingress	Routage HTTP externe par nom d'hôte	`fraud.local`, `api.fraud.local`
ConfigMap	Configuration non sensible	`backend-config`
Secret	Données sensibles (base64, chiffrable)	`fraud-api-secrets`, `alertmanager-...`
ServiceMonitor	Déclare une cible à Prometheus (CRD)	`fraud-backend-monitor`
PrometheusRule	Déclare des règles d'alerte (CRD)	`fraud-detection-alerts`

Les deux derniers sont des **CRD** (*Custom Resource Definitions*) apportés par l'opérateur Prometheus : Kubernetes est extensible, et un outil peut y ajouter ses propres types d'objets.

Ce que nous déployons



Sept manifestes pour l'application, quatre pour la supervision. Le namespace **isole** l'application : on peut tout supprimer d'un `kubectl delete namespace fraud-detection`. Le frontend n'a pas de ConfigMap : son URL d'API est figée au *build* de l'image.

Le Deployment backend

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: fraud-backend, namespace: fraud-detection, labels: {app: fraud-backend}}
spec:
  replicas: 1
  selector: {matchLabels: {app: fraud-backend}}   # <- doit matcher les labels du template
  template:
    metadata: {labels: {app: fraud-backend}}      # <- et le Service selectionne ces labels
    spec:
      containers:
        - name: fraud-backend
          image: <compte>/fraud-backend:latest   # puis <build>-<sha> via kubectl set image
          imagePullPolicy: Always
          ports: [{containerPort: 8000}]
          envFrom: [{configMapRef: {name: backend-config}}]
          env:                                   # jeton admin, si le Secret existe
            - {name: API_ADMIN_TOKEN, valueFrom: {secretKeyRef:
                {name: fraud-api-secrets, key: API_ADMIN_TOKEN, optional: true}}}
          livenessProbe:  {httpGet: {path: /live,   port: 8000}, initialDelaySeconds: 40}  # echec -> redemarrage
          readinessProbe: {httpGet: {path: /health, port: 8000}, initialDelaySeconds: 20}  # 503 -> hors du Service
          resources:
            requests: {memory: "256Mi", cpu: "250m"}
            limits:   {memory: "512Mi", cpu: "500m"}
```

Le manifeste versionné part de `:latest` ; le pipeline de déploiement pose ensuite un **tag immuable** `<build>-<sha>`, ce qui modifie la spec et déclenche un vrai *rolling update*, traçable dans `rollout history`.

Les trois sondes de santé

Sonde	Question	Si elle échoue	Notre réglage
`startupProbe`	Le démarrage est-il terminé ?	Les autres sondes attendent	⊗ non définie
`livenessProbe`	Le processus est-il vivant ?	**Le conteneur redémarre**	`/live`, 40 s, toutes les 30 s
`readinessProbe`	Peut-il recevoir du trafic ?	**Retiré du Service**	`/health` (503), 20 s, toutes les 10 s

POURQUOI 40 SECONDES

Le chargement du modèle et des processeurs prend une dizaine de secondes. Avec un `initialDelaySeconds` trop court, la *livenessProbe* échouerait pendant le démarrage et le pod entrerait en **boucle de redémarrage infinie**.

Corrigé : deux endpoints, deux questions

`/live` répond toujours ; `/health` renvoie **`503`** tant que le modèle n'est pas chargé. Un artefact manquant retire donc le pod du trafic *sans* boucle de redémarrages. Reste possible : une `startupProbe` pour les démarrages très longs.

Requests et limits : le contrat de ressources

DEUX NOTIONS DISTINCTES

- **requests** — ce que le pod **réserve**. Sert au *scheduler* pour choisir un nœud.
- **limits** — le **plafond**. Au-delà, le conteneur est bridé (CPU) ou tué (mémoire).

NOS VALEURS

Backend : 256 Mi / 250 m réservés, 512 Mi / 500 m au plafond.
Frontend : 128 Mi / 125 m réservés, 256 Mi / 250 m au plafond.

Le risque `OOMKilled`

Dépasser la limite mémoire ne ralentit pas le conteneur : il est **tué immédiatement** par le noyau. Or LightGBM, pandas et un `predictCSV` sur un gros fichier peuvent facilement dépasser 512 Mi. Symptôme : redémarrages inexpliqués, motif `OOMKilled` dans `kubectl describe pod`.

Méthode : mesurer avec `kubectl top pod` sous charge réelle, puis fixer `requests` au percentile 50 et `limits` au percentile 99 majoré.

ConfigMap et Secret

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
  namespace: fraud-detection
data:
  MODEL_REGISTRY_DIR: "/app/model_registry"
  PROCESSORS_DIR: "/app/artifacts/preprocessing"
  PYTHONUNBUFFERED: "1"
```

```bash
# Le Secret n'est PAS versionne : on le cree
kubectl create secret generic fraud-api-secrets \
  -n fraud-detection \
  --from-literal=API_ADMIN_TOKEN="$(openssl rand -hex 32)"
```

CONSOMMATION DANS LE POD

`envFrom: [configMapRef: {name: backend-config}]` injecte **toutes les clés** comme variables d'environnement ; `secretKeyRef` en injecte **une seule**, depuis un Secret.

CONFIGMAP OU SECRET

Même mécanisme, mais un `Secret` est encodé en base64, peut être chiffré au repos et n'apparaît pas dans les journaux d'audit standard. **Base64 n'est pas du chiffrement** : un Secret non chiffré est lisible par quiconque a accès à `etcd`.

> Modifier un ConfigMap **ne redémarre pas** les pods : les variables d'environnement sont lues au démarrage. Il faut `kubectl rollout restart deployment/...`.

Services : les trois types

```yaml
kind: Service
metadata:
  name: fraud-backend-service
spec:
  selector: {app: fraud-backend}   # <- lie le Service aux pods
  ports:
    - name: http                   # <- nomme : requis par le
        ServiceMonitor
      port: 8000
      targetPort: 8000
      nodePort: 30080
  type: NodePort
```

Le champ `name: http` du port n'est pas décoratif : le `ServiceMonitor` de Prometheus **référence le port par son nom**.

Type	Portée
`ClusterIP`	Interne au cluster uniquement (défaut). Notre frontend.
`NodePort`	Ouvre un port sur *chaque* nœud (30000–32767). Notre backend.
`LoadBalancer`	Demande un répartiteur au fournisseur cloud.

Le `NodePort 30080` permet à Jenkins d'atteindre l'API depuis l'extérieur du cluster (`http://192.168.49.2:30080`) pour publier les métriques de dérive. Pratique en local, à remplacer par un **nom DNS interne** en production.

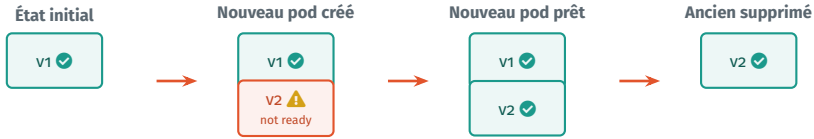
L'Ingress : un point d'entrée HTTP unique

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: fraud-ingress
  namespace: fraud-detection
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "50m"   # pour predictCSV
spec:
  rules:
    - host: fraud.local                    # l'interface
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: {service: {name: fraud-frontend-service, port: {number: 3000}}}
    - host: api.fraud.local                # l'API
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: {service: {name: fraud-backend-service, port: {number: 8000}}}
```

`proxy-body-size: 50m` : sans cette annotation, NGINX rejette les téléversements CSV au-delà de **1 Mo** par défaut.

En local, il faut ajouter dans `/etc/hosts` :
`192.168.49.2 fraud.local api.fraud.local`

Le rolling update en détail



le trafic n'est envoyé au pod v2 qu'après succès de la readinessProbe

LES DEUX PARAMÈTRES CLÉS

`maxSurge` — combien de pods en plus pendant la transition.
`maxUnavailable` — combien de pods indisponibles tolérés.
Avec `replicas: 1` et les valeurs par défaut (25 %), la transition reste **séquentielle**.

Avec **une seule réplique**, une brève indisponibilité est possible. Pour un service critique : `replicas: 2` minimum, plus un `PodDisruptionBudget`.

Les commandes `kubectl` du quotidien

```
# --- Deployer ---
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/backend/          # applique tout un repertoire

# --- Observer ---
kubectl get pods -n fraud-detection -w            # -w : suivre en direct
kubectl get all -n fraud-detection
kubectl describe pod <nom> -n fraud-detection     # evenements : LA source de diagnostic
kubectl logs -f deployment/fraud-backend -n fraud-detection
kubectl logs <pod> --previous -n fraud-detection  # journaux du conteneur PRECEDENT
kubectl top pods -n fraud-detection               # CPU / memoire consommes

# --- Mettre a jour et revenir en arriere ---
kubectl set image deployment/fraud-backend fraud-backend=<image>:<build>-<sha> -n fraud-detection
kubectl rollout status  deployment/fraud-backend -n fraud-detection --timeout=300s
kubectl rollout history deployment/fraud-backend -n fraud-detection
kubectl rollout undo    deployment/fraud-backend -n fraud-detection   # <- ROLLBACK
kubectl rollout restart deployment/fraud-backend -n fraud-detection

# --- Deboguer ---
kubectl exec -it <pod> -n fraud-detection -- /bin/bash
kubectl port-forward svc/fraud-backend-service 8000:8000 -n fraud-detection
```

`kubectl logs –previous` est la commande la plus utile face à un `CrashLoopBackOff` : elle montre **pourquoi** le conteneur est mort.

Diagnostiquer un pod qui ne démarre pas

État	Signification	Causes fréquentes
`Pending`	Aucun nœud ne peut accueillir le pod	`requests` trop élevés, nœud saturé
`ImagePullBackOff`	L'image ne peut être téléchargée	Nom ou tag erroné, registre privé sans secret
`CrashLoopBackOff`	Le conteneur démarre puis meurt en boucle	Exception au démarrage, **fichier modèle absent**, *livenessProbe* trop agressive
`OOMKilled`	Limite mémoire dépassée	`limits.memory` trop bas
`Running` mais `0/1`	Pod vivant mais **pas prêt**	*readinessProbe* en échec
`Error` / `Completed`	Le processus principal s'est terminé	`CMD` qui rend la main (pas un serveur)

Méthode de diagnostic en trois commandes

1. `kubectl get pods` — quel est l'état ?
2. `kubectl describe pod <nom>` — section *Events* en bas : la cause y est presque toujours écrite.
3. `kubectl logs <nom> –previous` — ce que l'application a dit avant de mourir.

Spécificités du ML sur Kubernetes

CONTRAINTES PROPRES AU ML

- **Images volumineuses** (600 Mo et plus) : le démarrage dépend du téléchargement.
- **Démarrage lent** : chargement du modèle en mémoire.
- **Mémoire élevée** : pandas et LightGBM sont gourmands.
- **Charge en rafales** : les lots CSV créent des pics.
- **État du modèle** : quelle version tourne dans quel pod ?

RÉPONSES POSSIBLES

- Images multi-stage, cache d'images sur les nœuds.
- `startupProbe` généreuse.
- `requests`/`limits` mesurés sous charge réelle.
- **HPA** sur le CPU ou sur une métrique métier (*KEDA* pour les métriques Prometheus).
- Version du modèle exposée comme **métrique** (`fraud_model_version_info`).

Débat classique : **modèle dans l'image** ou **modèle sur un volume** ? Dans l'image, on obtient l'immuabilité et l'autonomie ; sur un volume, on change de modèle sans reconstruire. Nous avons choisi l'image, parce que la reproductibilité prime et que le pipeline reconstruit de toute façon.

Bonnes pratiques Kubernetes

À faire

- Un namespace dédié par application.
- Toujours définir `requests` **et** `limits`.
- Toujours définir les sondes de santé.
- Étiqueter les objets de façon cohérente.
- Versionner les manifestes dans Git.
- `replicas` ≥ 2 pour un service critique.
- Vérifier le *rollout* après chaque déploiement.
- Utiliser des images taguées par SHA.

À éviter

- Déployer dans le namespace `default`.
- Créer des pods « nus », sans Deployment.
- Modifier un objet avec `kubectl edit` en production (l'écart avec Git devient invisible).
- Stocker un secret en clair dans un manifeste versionné.
- Déployer `:latest` : `set image` ne change rien et le *rollback* n'a pas de version cible.
- Omettre les `limits` : un pod peut asphyxier son nœud.
- Confondre *liveness* et *readiness*.

Étape suivante naturelle : le **GitOps** (Argo CD, Flux), où le cluster se synchronise **lui-même** sur le dépôt Git, au lieu que Jenkins y pousse des commandes.

13

Monitoring : Prometheus, Grafana et Evidently AI

Pourquoi un modèle doit être surveillé

LE MODE DE DÉFAILLANCE PROPRE AU ML

Un logiciel classique tombe : on le voit. Un modèle, lui, **continue de répondre** — il répond simplement de plus en plus mal. Aucune exception, aucune erreur 500, aucun redémarrage.

CAUSES DE DÉGRADATION

- Les fraudeurs changent de mode opératoire (*concept drift*).
- Le comportement des clients évolue (*data drift*).
- Une source amont change de format ou d'unité.
- Une nouvelle catégorie de commerçant apparaît.
- Un bug dans le pipeline de features.

Le scénario que le monitoring évite

1. Semaine 1 — le modèle détecte 87 % des fraudes.
2. Semaine 12 — il en détecte 60 %, sans que rien ne l'indique.
3. Semaine 20 — le service des litiges signale une hausse anormale des réclamations.
4. Semaine 21 — on découvre que la distribution des montants a changé depuis trois mois.

Objectif du monitoring : passer de **« on l'apprend par le métier après 5 mois »** à **« une alerte se déclenche en 48 heures »**.

Les quatre couches à surveiller

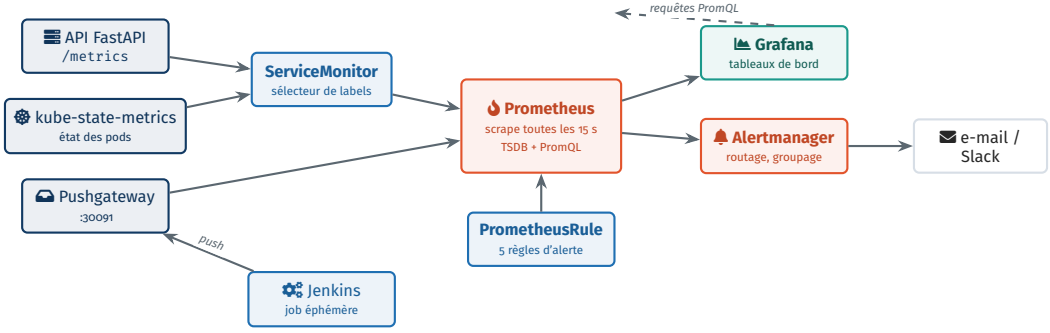
Couche	Ce qu'on mesure	Exemples	Outil
Infrastructure	Santé des ressources	CPU, RAM, pods, redémarrages	Prometheus + kube-state
Service	Comportement de l'API	Latence, débit, taux d'erreur	Prometheus (client Python)
Métier	Effets observables	Taux de fraude, volume d'alertes	Métriques applicatives
Modèle	Validité statistique	Dérive des entrées, des prédictions	**Evidently AI**

L'ERREUR CLASSIQUE

Ne surveiller que les deux premières couches. Un service peut afficher **100 % de disponibilité et 20 ms de latence** tout en produisant des prédictions devenues inutiles.

> Les trois piliers de l'observabilité — **métriques, journaux, traces** — restent valables. Le ML y ajoute une quatrième famille : les **métriques statistiques**.

L'architecture de monitoring déployée



Prometheus fonctionne en mode **pull** : c'est **lui** qui interroge les cibles, à intervalle régulier. Les cibles n'ont qu'à exposer un endpoint HTTP.

Les quatre types de métriques

Type	Comportement	Usage	Exemple chez nous
Counter	Croît uniquement, remis à 0 au redémarrage	Volumes, erreurs	`fraud_predictions_total`
Gauge	Monte et descend librement	États, taux, scores	`fraud_rate_current`
Histogram	Compte par intervalles prédéfinis	Latences, tailles	`fraud_prediction_latency_seconds`
Summary	Quantiles calculés côté client	Rarement nécessaire	non utilisé

```
# Ce que renvoie GET /metrics  (format OpenMetrics)
# HELP fraud_predictions_total Total des predictions par resultat
# TYPE fraud_predictions_total counter
fraud_predictions_total{result="fraud"} 142.0
fraud_predictions_total{result="legitimate"} 8931.0
# TYPE fraud_prediction_latency_seconds histogram
fraud_prediction_latency_seconds_bucket{le="0.05"} 8402.0
fraud_prediction_latency_seconds_bucket{le="0.1"}  9011.0
fraud_prediction_latency_seconds_bucket{le="+Inf"} 9073.0
fraud_prediction_latency_seconds_sum   318.4
fraud_prediction_latency_seconds_count 9073.0
```

Les `{result="fraud"}` sont des **labels** : une même métrique porte plusieurs séries temporelles. Attention à la cardinalité — un label contenant un identifiant de transaction ferait exploser la base.

PromQL — les requêtes utiles pour notre cas

```
# 1. Debit de predictions par seconde, sur 5 min
rate(fraud_predictions_total[5m])

# 2. Repartition fraude / legitime
sum by (result) (rate(fraud_predictions_total[5m]))

# 3. Taux de fraude calcule (independant de la gauge applicative)
  sum(rate(fraud_predictions_total{result="fraud"}[5m]))
/ sum(rate(fraud_predictions_total[5m]))

# 4. Latence P99 -- LA requete a connaitre pour un histogramme
histogram_quantile(0.99, rate(fraud_prediction_latency_seconds_bucket[5m]))

# 5. Latence moyenne
  rate(fraud_prediction_latency_seconds_sum[5m])
/ rate(fraud_prediction_latency_seconds_count[5m])

# 6. Taux d'erreur d'inference
rate(fraud_prediction_errors_total[5m]) / rate(fraud_predictions_total[5m])

# 7. Redemarrages de pods sur 5 min
increase(kube_pod_container_status_restarts_total{namespace="fraud-detection"}[5m])
```

`rate()` sur un *counter* et `histogram_quantile()` sur les *buckets* sont les deux fonctions à maîtriser : elles couvrent **90 % des besoins** réels.

Comment Prometheus découvre notre API

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: fraud-backend-monitor
  namespace: monitoring          # <- ou vit Prometheus
  labels:
    release: monitoring          # <- capte par l'operateur
spec:
  selector:
    matchLabels:
      app: fraud-backend         # <- selectionne le SERVICE
  namespaceSelector:
    matchNames:
      - fraud-detection          # <- dans CET autre namespace
  endpoints:
    - port: http                 # <- NOM du port du Service
      path: /metrics
      interval: 15s
```

LA CHAÎNE DE SÉLECTION

`Prometheus` → (label `release: monitoring`) → `ServiceMonitor` → (label `app: fraud-backend`) → `Service` → (selector) → `Pods`.

Les trois erreurs qui font échouer la découverte

1. Le label `release` ne correspond pas à celui attendu par l'installation de Prometheus.
2. Le port du Service **n'est pas nommé** : le `ServiceMonitor` le référence par son nom.
3. Le `namespaceSelector` est omis alors que l'application vit dans un autre namespace.

Vérification : page *Status* → *Targets* de Prometheus. La cible doit apparaître à l'état `UP`.

Le Pushgateway : surveiller un job éphémère

```python
# jenkins/push_metrics.py -- appele dans le bloc post du pipeline
PUSHGATEWAY_URL = os.getenv("PUSHGATEWAY_URL", "http://192.168.49.2:30091")
def build_payload(build_status):
    body = gauge("fraud_jenkins_build_status", "Statut du dernier build", build_status)
    # + fraud_ci_scenario_* (scenario_test.json), fraud_ci_model_promoted (promotion.json)
    return body
requests.post(f"{PUSHGATEWAY_URL}/metrics/job/jenkins_ml_pipeline",
              data=build_payload(build_status), timeout=5)
```

POURQUOI UN COMPOSANT DÉDIÉ

Prometheus fonctionne en **pull**. Or un pipeline Jenkins dure trois minutes : quand Prometheus voudrait l'interroger, il n'existe plus. Le **Pushgateway** sert de **tampon persistant** : le job y **pousse** sa métrique, Prometheus la récupère plus tard.

Le Pushgateway est à réserver aux **jobs par lots**. L'utiliser pour un service permanent est un anti-pattern : la métrique **ne disparaît pas** quand le service meurt, ce qui masque les pannes.

Résultat : la santé du **pipeline CI/CD** s'affiche à côté de celle de l'application ; l'alerte `MLPipelineFailed` en fait un incident.

Nos cinq règles d'alerte

```yaml
- alert: HighFraudRate                          # 1. anomalie METIER (calculee par PromQL)
  expr: ( sum(increase(fraud_predictions_total{result="fraud"}[10m]))
          / sum(increase(fraud_predictions_total[10m])) ) > 0.3
        and sum(increase(fraud_predictions_total[10m])) >= 20     # volume minimal
  for: 2m
  labels: {severity: critical}

- alert: DriftDetected                          # 2. anomalie STATISTIQUE
  expr: fraud_drift_detected == 1
  for: 0m
  labels: {severity: warning}

- alert: HighLatency                            # 3. anomalie de SERVICE
  expr: histogram_quantile(0.99, rate(fraud_prediction_latency_seconds_bucket[5m])) > 2
  for: 2m
  labels: {severity: warning}

- alert: PodCrash                               # 4. anomalie d'INFRASTRUCTURE
  expr: increase(kube_pod_container_status_restarts_total{namespace="fraud-detection"}[5m]) > 0
  labels: {severity: critical}

- alert: MLPipelineFailed                       # 5. anomalie de la CHAINE CI/CD
  expr: fraud_jenkins_build_status == 0         # pousse par Jenkins via le Pushgateway
  labels: {severity: warning}
```

Le champ `for:` évite les fausses alertes : la condition doit rester vraie **pendant toute la durée indiquée**. `for: 0m` (valeur par défaut) signifie « alerter immédiatement » — justifié pour un crash, discutable pour une dérive.

Concevoir une bonne alerte

Une alerte utile

- Signale un problème **réel** et **actionnable**.
- Indique **quoi faire** dans son annotation.
- A un seuil justifié par des mesures, pas par intuition.
- Utilise `for:` pour filtrer le bruit.
- A une sévérité qui détermine le canal (*critical* → astreinte, *warning* → e-mail).

La fatigue d'alerte

Une alerte qui se déclenche dix fois par jour sans nécessiter d'action est **pire que pas d'alerte** : l'équipe apprend à l'ignorer, puis ignore aussi les vraies. Test à s'appliquer : **« si cette alerte sonne à 3 h du matin, quelqu'un doit-il se lever ? »** Si la réponse est non, ce n'est pas une alerte : c'est un panneau de tableau de bord.

La première version, `fraud_rate_current > 0.3`, était **ambiguë** : la jauge reflétait la *dernière requête*, donc un seul appel sur une transaction frauduleuse la portait à 100 %. Elle est désormais calculée par PromQL **sur 10 minutes, avec au moins 20 prédictions**. Reste à distinguer trafic de production et trafic de test, par un label.

Grafana : du chiffre au tableau de bord

LE RÔLE DE GRAFANA

Prometheus **stocke et calcule** ; Grafana **montre et raconte**.

- Se branche sur Prometheus comme *data source*.
- Un *dashboard* rassemble des *panels*.
- Chaque *panel* exécute une requête PromQL.
- Les variables (`$namespace`) rendent le dashboard réutilisable.
- Le JSON du dashboard **doit être versionné dans Git**.

CHOISIR LA BONNE VISUALISATION

- **Time series** — évolution d'une métrique (débit, latence).
- **Stat** — une valeur unique et son seuil (taux de fraude).
- **Gauge** — une valeur dans une plage (score de dérive).
- **Bar gauge** — comparaison entre catégories.
- **Table** — liste d'états (pods, versions).
- **State timeline** — historique d'un état binaire (dérive détectée, build OK).

> Règle de conception : un tableau de bord doit répondre à **une question**. « L'application est-elle en bonne santé ? » et « le modèle est-il encore valide ? » sont deux questions, donc **deux tableaux de bord**.

Le tableau de bord MLOps de la plateforme

Rangée	Panneau	Requête	Lecture
Métier	Taux de fraude (stat)	`fraud_rate_current`	Rouge au-delà de 0,3
Métier	Prédictions par classe (série)	`rate(fraud_predictions_total[5m])`	Volume et répartition
Service	Latence P50/P95/P99	`histogram_quantile(...)`	Respect de la contrainte
Service	Taux d'erreur	`rate(fraud_prediction_errors_total[5m])`	Doit rester à zéro
Modèle	Score de dérive (jauge)	`fraud_drift_score`	Seuil à 0,30
Modèle	Features en dérive	`fraud_drift_features_count`	Sur 6 surveillées
Modèle	Version servie (table)	`fraud_model_version_info`	Traçabilité
Infra	Redémarrages de pods	`increase(...restarts_total[1h])`	Signale l'instabilité
Infra	Statut du pipeline	`fraud_jenkins_build_status`	1 = dernier build OK

Quatre rangées, quatre couches : un coup d'œil suffit à situer un problème — **métier, service, modèle ou infrastructure**.

Evidently AI — les types de dérive

Data drift
$P(X)$ change
la distribution des entrées bouge
ex. : montants moyens en hausse

Target drift
$P(y)$ change
le taux de fraude réel bouge
ex. : campagne d'attaque

Concept drift
$P(y|X)$ change
la *relation* bouge
ex. : nouveau mode opératoire

Prediction drift
$P(\hat{y})$ change
les sorties du modèle bougent
ex. : plus d'alertes

CE QU'ON PEUT MESURER SANS ÉTIQUETTES

✅ **Data drift** et **prediction drift** : ils ne nécessitent que les entrées et les sorties. ❌ **Target** et **concept drift** exigent la **vérité terrain**, qui arrive avec des semaines de retard.

D'où notre stratégie : surveiller le **data drift** comme **signal avancé**. Ce n'est pas une preuve de dégradation, mais c'est le meilleur indicateur disponible *immédiatement*.

Comment on mesure statistiquement une dérive

Test	Type de variable	Principe	Seuil usuel
Kolmogorov–Smirnov	Numérique	Écart maximal entre fonctions de répartition	p-value $< 0{,}05$
Wasserstein	Numérique	« Coût de transport » entre les deux distributions	distance $> 0{,}1$
Population Stability Index	Numérique (binné)	Somme pondérée des écarts par intervalle	PSI $> 0{,}2$
Chi-deux	Catégorielle	Écart entre fréquences observées et attendues	p-value $< 0{,}05$
Jensen–Shannon	Catégorielle	Divergence entre distributions discrètes	distance $> 0{,}1$

CE QUE FAIT EVIDENTLY

Il **choisit le test automatiquement** selon le type de la colonne et la taille de l'échantillon : tests d'hypothèse sur les petits échantillons, mesures de distance sur les grands — car avec 600 000 lignes, **tout écart devient statistiquement significatif**.

C'est le **piège du grand échantillon** : une p-value de 0,001 sur 600 000 lignes peut correspondre à un écart de moyenne de 0,2 %, sans aucune conséquence sur le modèle. Les **mesures de distance** sont préférables à cette échelle.

Notre détection de dérive

```
FEATURES        = ['amt', 'category', 'gender', 'city_pop', 'merch_lat', 'merch_long']
DRIFT_THRESHOLD = float(os.getenv("DRIFT_THRESHOLD", "0.3"))

reference_df = pd.read_csv("data/raws/fraud_600k.csv", usecols=FEATURES)  # la reference
new_df       = pd.read_csv("data/raws/new_data.csv",   usecols=FEATURES)  # les nouvelles

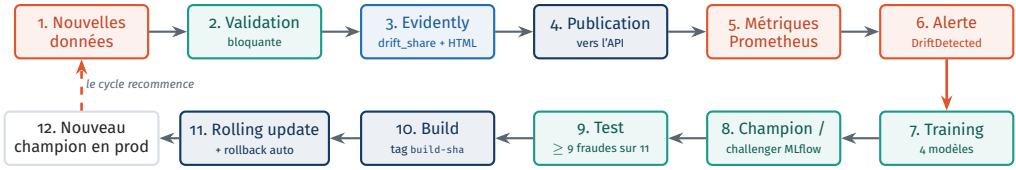
report  = Report([DataDriftPreset()])
my_eval = report.run(new_df, reference_df)
my_eval.save_html("artifacts/jenkins/drift_report.html")           # rapport interactif

for metric in my_eval.dict().get('metrics', []):                   # extraction du resultat
    if 'DriftedColumnsCount' in metric.get('metric_name', ''):
        drift_share = metric['value'].get('share', 0)              # part de colonnes en derive
        drifted     = int(metric['value'].get('count', 0))

drift_detected = drift_share > DRIFT_THRESHOLD
write_outputs(drift_detected, drift_share, drifted)   # -> drift_result.txt + drift_summary.json
# L'envoi a l'API est un stage distinct (publish_drift.py, en-tete X-API-Token) :
# le script de mesure ne depend plus de la disponibilite de l'API.
```

Avec 6 features surveillées, un seuil de 0,30 signifie concrètement : **2 features en dérive ou plus déclenchent le ré-entraînement** ($2/6 = 0{,}33$). Un seuil doit toujours être traduit en nombre concret pour être discuté.

La boucle complète : de la dérive au nouveau modèle



Douze étapes, **zéro intervention humaine** quand tout va bien, et **quatre points d'arrêt** quand quelque chose cloche : données invalides, challenger moins bon, test de scénario raté, rollout en échec. Le monitoring ne se contente pas d'*informer*, il **agit** — et la supervision humaine porte sur **la boucle elle-même**.

Les limites de notre dispositif

Limites à assumer

- **Pas de vérité terrain** : on ne mesure jamais le rappel réel en production.
- **Référence figée** : `fraud_600k.csv` vieillit ; une dérive lente et légitime finira par la faire diverger.
- **Six features surveillées** sur 21 : une dérive sur `distance_km` passerait inaperçue.
- **Dérive par lot**, pas en continu : on compare des fichiers, pas le trafic réel.
- **Ré-entraînement sur l'ancien jeu** : le CT relit `preprocessed_data.pkl`, pas `new_data.csv` — la dérive *déclenche* l'entraînement sans en changer les données.

Améliorations : faites et à faire

- ✅ Valider les données **avant** de ré-entraîner (stage 3).
- ✅ Exiger que le challenger **batte** le champion.
- ✅ *Rollback* automatique si le rollout échoue.

1. Ré-entraîner sur référence + nouvelles données.
2. Journaliser les prédictions réelles et les comparer aux litiges quand ils arrivent.
3. Étendre la surveillance aux features dérivées.
4. Faire glisser la fenêtre de référence.
5. Ajouter un déploiement canari.

La validation était la priorité : **un ré-entraînement automatique sans validation des données est une arme pointée vers soi-même**.

Bonnes pratiques de monitoring ML

À faire

- Instrumenter dès le premier jour, pas après le premier incident.
- Surveiller les quatre couches, pas seulement l'infrastructure.
- Exposer la **version du modèle** comme métrique.
- Conserver les rapports de dérive (traçabilité, audit).
- Définir les seuils avec le métier.
- Versionner les tableaux de bord Grafana.
- Tester les alertes (les déclencher volontairement).

À éviter

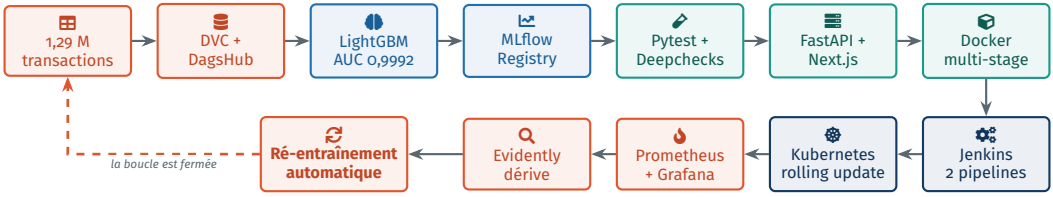
- Des labels à cardinalité élevée (id de transaction, timestamp).
- Alerter sur tout, donc n'être alerté sur rien.
- Un tableau de bord de 40 panneaux illisible.
- Confondre « le service répond » et « le modèle est bon ».
- Utiliser le Pushgateway pour un service permanent.
- Ré-entraîner automatiquement sans valider les données.
- Laisser une alerte se déclencher sans destinataire.

Question de contrôle : **si le modèle se dégradait aujourd'hui, combien de temps avant que quelqu'un l'apprenne ?** La réponse mesure la qualité réelle du monitoring.

14

Synthèse et feuille de route

Le chemin parcouru



Onze technologies, un seul objectif : qu'un modèle de détection de fraude reste **utile, disponible et à jour** sans qu'un humain doive intervenir chaque semaine.

Chaque outil répond à un problème précis

Outil	Problème qu'il résout	Sans lui
Git	Historiser et collaborer sur le code	Des dossiers `projet_v3_final`
DVC	Versionner des données volumineuses	Dépôt inutilisable, données introuvables
MLflow	Comparer et gouverner les modèles	« Quel modèle est en prod ? » sans réponse
FastAPI	Rendre le modèle consommable	Un `.pkl` utilisable par une seule personne
Next.js	Rendre le service utilisable par le métier	Un `curl` réservé aux techniciens
Pytest	Empêcher les régressions	Des bugs découverts en production
Deepchecks	Détecter les données invalides	Un modèle entraîné sur des données corrompues
Docker	Reproduire l'environnement	« Ça marche sur ma machine »
Jenkins	Automatiser la chaîne complète	Deux heures de manipulations par livraison
Kubernetes	Disponibilité et mise à jour sans coupure	Interruption à chaque déploiement
Prometheus	Mesurer le comportement réel	Aucune visibilité
Grafana	Rendre les mesures lisibles	Des métriques que personne ne consulte
Evidently	Détecter la péremption du modèle	Une dégradation silencieuse pendant des mois

Dix enseignements à retenir

1. Le code d'entraînement représente **moins de 10 %** du travail réel.
2. Un modèle en production n'est pas un fichier, c'est un **service instrumenté**.
3. Ce qu'on livre n'est pas un modèle, c'est le **pipeline qui le produit**.
4. Le **training–serving skew** est la première cause d'échec : sérialisez les processeurs.
5. L'accuracy est trompeuse sur données déséquilibrées ; le **rappel** parle au métier.
6. Un score hors ligne trop beau est un **symptôme**, pas un succès.
7. L'automatisation sans **quality gates** ne fait qu'accélérer les erreurs.
8. Un test qui ne peut pas **bloquer** la chaîne n'est qu'un rapport.
9. Le monitoring ML est **statistique** avant d'être technique.
10. Le bon niveau de maturité est celui **qu'exige l'enjeu**, pas le plus élevé.

Nos six priorités d'amélioration — et où elles en sont

	Action	Pourquoi	État dans cette version
1	Valider les données **avant** le ré-entraînement	On pouvait ré-entraîner sur des données corrompues	✅ `validate_data.py`, stage bloquant
2	Durcir le test de scénario : `detected >= 9`	Le test ne bloquait aucune régression	✅ quality gate `MIN_DETECTED`
3	Renvoyer `503` sur `/health` quand le modèle est absent	La *readinessProbe* ne protégeait rien	✅ `/health` (503) + `/live`
4	Taguer les images par build et commit	`:latest` rendait la traçabilité impossible	✅ `<build>-<sha>`, rollback auto
5	Exiger que le candidat batte le champion	On pouvait promouvoir un modèle moins bon	✅ alias `champion`, `val_pr_auc`
6	Sortir les secrets des fichiers versionnés et les révoquer	Un jeton dans le dépôt est un jeton compromis	⚠️ retiré du dépôt ; **révocation à faire**

Et maintenant

Les cinq premières ont demandé **peu de code** et corrigent les faiblesses les plus structurantes. La sixième ne se règle pas dans le code : un secret publié reste dans l'**historique Git** et doit être **révoqué**. Prochaines étapes : ré-entraîner sur les nouvelles données, brancher Deepchecks en CI, `replicas: 2` et déploiement canari.

Liste de contrôle avant mise en production

DONNÉES ET MODÈLE

- ☐ Dataset versionné et retrouvable par son hash.
- ☐ Split effectué avant toute transformation.
- ☐ Processeurs sérialisés et réutilisés à l'inférence.
- ☐ Métrique de sélection validée par le métier.
- ☐ Seuil de décision explicite et paramétrable.
- ☐ Run MLflow retrouvable depuis la production.

CODE ET TESTS

- ☐ Aucun secret dans le dépôt.
- ☐ Versions de dépendances épinglées.
- ☐ Tests unitaires et d'intégration en CI.
- ☐ Validation des données bloquante.

DÉPLOIEMENT

- ☐ Image reproductible et taguée par version.
- ☐ Conteneur en utilisateur non-root.
- ☐ Sondes de vie et de disponibilité configurées.
- ☐ `requests` et `limits` mesurés.
- ☐ Procédure de rollback testée **en vrai**.

EXPLOITATION

- ☐ Métriques techniques, métier et de dérive exposées.
- ☐ Tableau de bord partagé avec le métier.
- ☐ Alertes avec destinataire identifié.
- ☐ Stratégie de ré-entraînement décidée.
- ☐ Procédure d'incident écrite.

Pour aller plus loin

BRIQUES SUIVANTES

- **Feature store** (Feast, Tecton) — partager les features entre entraînement et service, avec agrégats temporels.
- **Serving spécialisé** (KServe, BentoML, Seldon) — *autoscaling*, canari, tests A/B, *batching*.
- **Orchestration de workflows** (Airflow, Kubeflow, Dagster) — graphes de tâches, reprise sur incident.
- **GitOps** (Argo CD, Flux) — le cluster se synchronise lui-même sur Git.

ENJEUX TRANSVERSES

- **Explicabilité** (SHAP, LIME) — obligation pratique en secteur bancaire pour justifier un refus.
- **Équité** — mesurer les disparités de traitement entre groupes.
- **Gouvernance** — documentation des modèles (*model cards*), pistes d'audit, conformité.
- **LLMOps** — versionnement de prompts, évaluation, coûts par requête, garde-fous.

> Aucune de ces briques ne remplace les fondations vues dans ce cours : elles s'y **ajoutent**. Un feature store sur un projet qui ne sait pas reproduire un entraînement ne résout rien.

Références

OUVRAGES

- N. Gift, A. Deza — *Practical MLOps*, O'Reilly, 2021.
- A. P. McMahon — *Machine Learning Engineering with Python*, Packt, 2023.
- V. Lakshmanan, S. Robinson, M. Munn — *Machine Learning Design Patterns*, O'Reilly, 2020.
- C. Huyen — *Designing Machine Learning Systems*, O'Reilly, 2022.

ARTICLES FONDATEURS

- D. Sculley et al. — *Hidden Technical Debt in Machine Learning Systems*, NeurIPS 2015.
- E. Breck et al. — *The ML Test Score*, IEEE Big Data 2017.

RÉFÉRENTIELS DE MATURITÉ

- Google Cloud — *MLOps : Continuous delivery and automation pipelines in machine learning.*
- Microsoft Azure — *Machine Learning operations maturity model.*

DOCUMENTATIONS

- `dvc.org/doc` — `mlflow.org/docs`
- `fastapi.tiangolo.com` — `docs.deepchecks.com`
- `docs.evidentlyai.com` — `prometheus.io/docs`
- `kubernetes.io/docs` — `jenkins.io/doc`

LE PROJET

Dépôt du cours : `README.md`, section « Guide complet ».

Merci de votre attention

MLOps — Plateforme de détection de fraude bancaire

De l'expérimentation au ré-entraînement automatique en production

Questions ?